**CAN THO UNIVERSITY**

**COLLEGE OF INFORMATION AND COMMUNICATION TECHNOLOGY**

**FACULTY OF SOFTWARE ENGINEERING**


## SOFTWARE DEVELOPMENT THESIS PROJECT

**COURSE: CT250H**

# DESIGN AND IMPLEMENTATION OF A HUMAN-IN-THE-LOOP AI-ASSISTED RECRUITMENT AUTOMATION PLATFORM FOR CV-JD EVALUATION AND RECOMMENDATION IN IT

Supervisor:
Ph.D. Nguyen Thanh Khoa

Student:
Pham Huu Hung – B2203557

Can Tho, August 2026

# ACKNOWLEDGEMENTS

I would like to express my sincere gratitude to Ph.D. Nguyen Thanh Khoa for the guidance, constructive feedback, and continuous support provided throughout this thesis. The supervisor's academic and technical advice played an important role in shaping the research direction and improving the quality of this work.

I am also grateful to the lecturers and staff of the College of Information and Communication Technology, Can Tho University, for providing the academic foundation and learning environment necessary for the completion of this project. My thanks are extended to everyone who contributed feedback during the design, implementation, and evaluation of the CareerFit system.

Finally, I would like to thank my family and friends for their encouragement and support throughout my studies. Although every effort has been made to ensure the accuracy and quality of this thesis, limitations may remain, and constructive comments are sincerely appreciated.

Can Tho, August 2026

Pham Huu Hung

# ABSTRACT

Recruitment in the information technology sector requires candidates and recruiters to process large amounts of heterogeneous information from curricula vitae and job descriptions. Conventional job portals primarily support posting, searching, and manual application workflows, while automated screening tools may provide limited transparency regarding how a score is produced or how an automated action is controlled. This thesis presents CareerFit IT AutoPilot, a web-based recruitment platform that combines a job portal, CV–job description matching, personalized job recommendation, policy-driven automation, and Human-in-the-Loop interaction in a unified workflow.

The system processes candidate profiles, curricula vitae, job descriptions, preferences, and user feedback. Textual information is normalized and represented with Term Frequency–Inverse Document Frequency vectors. Cosine similarity ranks CV–job pairs and candidate–job recommendations, while the Rocchio relevance-feedback method updates learned representations from explicit feedback. An AutoFit policy layer evaluates scores together with consent, thresholds, interaction history, quotas, cooldown rules, time zones, and quiet hours before selecting an action. Expiring email-action links use hashed token storage, a non-mutating confirmation request, and POST redemption. Audit records and action states support traceability and replay prevention.

CareerFit is implemented as a role-based web platform for guests, candidates, recruiters, and administrators using a Spring Boot backend, a React and TypeScript frontend, and PostgreSQL persistence managed through Flyway migrations. The refreshed backend suite passed 72 of 72 tests, the production frontend build completed with a largest JavaScript chunk of 378.44 kB, and all 20 Chromium workflow, contract, and resilience tests passed. On a synthetic causal benchmark containing 50 Jobs and 100 CVs, Rocchio increased nDCG@5 from 0.037737 to 0.837737. The final benchmark log contained no optimistic-lock exception, and aggregate runtime health returned HTTP 200 with status UP. These results demonstrate controlled feedback behavior and integrated prototype workflows, not production hiring effectiveness.

The project demonstrates a practical approach to recruitment automation in which explainable scoring and configurable policies assist users without removing human oversight from consequential decisions. Its current limitations include the use of lexical vector representations and controlled evaluation data; semantic representations, larger real-world studies, and extended governance mechanisms are identified as future work.
*Keywords: recruitment automation; CV–job matching; recommendation system; Human-in-the-Loop; Rocchio feedback; explainable automation.*

# TABLE OF CONTENTS

# LIST OF FIGURES

# LIST OF TABLES

# LIST OF ABBREVIATIONS

| Abbreviation | Full Term | Meaning / Explanation |
|---|---|---|
| AI | Artificial Intelligence | Computer systems designed to perform tasks commonly associated with human intelligence |
| API | Application Programming Interface | A defined interface through which software components communicate |
| CV | Curriculum Vitae | A document describing a candidate's qualifications and experience |
| E2E | End-to-End | Testing of a complete workflow across integrated components |
| HITL | Human-in-the-Loop | Automation in which humans retain oversight or approval authority |
| JD | Job Description | A description of a job's responsibilities and requirements |
| JWT | JSON Web Token | A compact token format used for authentication and authorization |
| MRR | Mean Reciprocal Rank | A ranking metric based on the position of the first relevant item |
| nDCG | Normalized Discounted Cumulative Gain | A ranking metric that accounts for relevance and position |
| NLP | Natural Language Processing | Methods for processing and analyzing human language |
| OCR | Optical Character Recognition | Extraction of machine-readable text from images or scanned documents |
| RBAC | Role-Based Access Control | Authorization based on assigned user roles |
| REST | Representational State Transfer | An architectural style for networked APIs |
| TF-IDF | Term Frequency–Inverse Document Frequency | A weighting method for representing text terms |
| UAT | User Acceptance Testing | Validation of whether the system satisfies intended user workflows |

# CHAPTER 1. INTRODUCTION

## 1.1 Problem Statement

IT recruitment requires candidates and recruiters to compare many details, including technical skills, experience, language, location, and working conditions. Candidates must identify suitable vacancies, while recruiters must review CVs and decide which applicants deserve closer attention. As the number of jobs and profiles grows, doing this manually becomes slow and inconsistent [10].

Most job portals support posting, searching, and applying, but they do not fully explain why a job fits a candidate or why one applicant ranks above another. CareerFit therefore treats CV–JD matching and personalized recommendation as related but separate tasks. The first compares a specific CV and job; the second prioritizes jobs from a candidate's broader profile and preferences.

Automation can reduce repetitive work, but a similarity score should not directly trigger a consequential action. Consent, current application state, previous interactions, thresholds, quotas, and exceptional conditions also matter. CareerFit uses a Human-in-the-Loop design so that users can review important actions and administrators can trace what happened.

This thesis develops CareerFit IT AutoPilot, an IT-focused web platform that combines job discovery, CV and JD processing, matching, recommendation, Rocchio feedback, AutoFit policies, email actions, and audit records. The aim is not to replace recruitment judgment, but to provide understandable evidence and controlled assistance throughout the workflow.

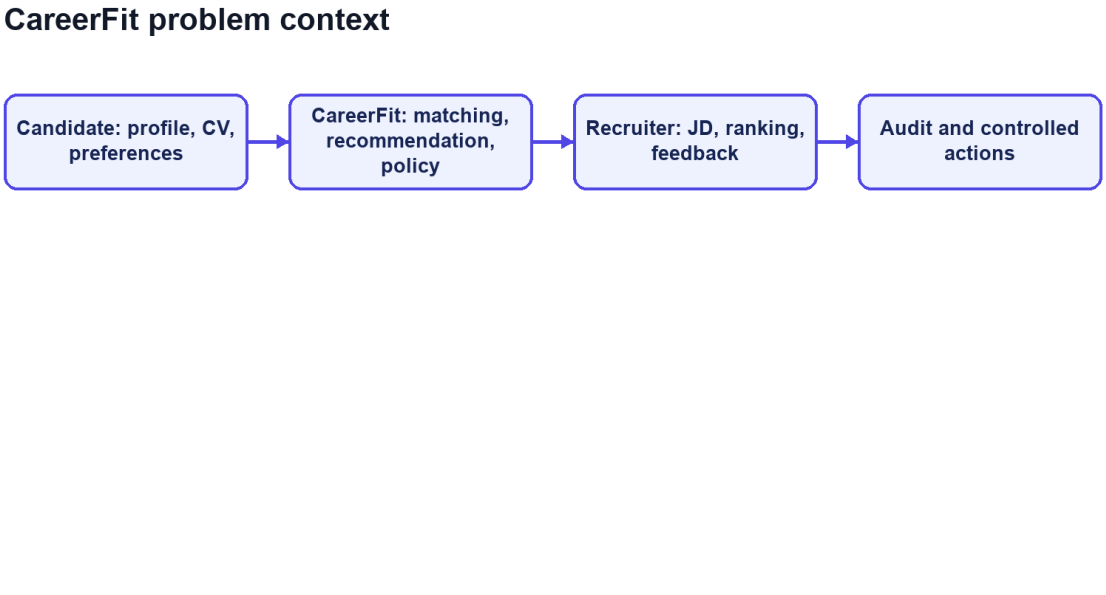



*Figure 1.1. CareerFit problem context and principal information flows*

## 1.2 Motivation

The first motivation is to reduce fragmentation. Candidates often move between job search, CV management, application history, and email, while recruiters switch between vacancy management, applicant review, candidate discovery, and reporting. CareerFit brings these activities into one role-based system and uses email as an additional action channel.

The second motivation is explainability. TF-IDF and cosine similarity do not capture meaning as deeply as modern embedding models, but their terms and weights can be inspected. This makes them suitable for an academic baseline in which the scoring process, limitations, and effect of feedback must be demonstrated clearly.

The third motivation is controlled adaptation and automation. AutoFit places policy checks between a score and an action, considering consent, thresholds, previous interactions, cooldown, quota, time zone, and quiet hours. Rocchio feedback then provides a transparent way to adjust later rankings from explicit positive and negative judgments without overwriting the original representation.

## 1.3 Objectives

### 1.3.1 Overall Objective

The overall objective of this thesis is to design, implement, and evaluate a web-based Human-in-the-Loop recruitment automation platform for the IT domain. The platform is intended to support CV–job description evaluation, personalized job recommendation, feedback-based adaptation, and policy-driven actions while preserving user control, explainability, security, and auditability.

### 1.3.2 Specific Objectives

To achieve the overall objective, the thesis defines the following specific objectives:

- Design a role-based web platform for guests, candidates, recruiters, and administrators, with appropriate public and authenticated workflows.
- Provide candidate profile and CV management, including document upload, form-based entry, text extraction, OCR fallback, validation, and quality feedback.
- Implement separate CV–JD matching and candidate-profile recommendation workflows using a shared text-normalization, TF-IDF, and cosine-similarity pipeline.
- Present normalized scores, relevance labels, matching reasons, validation signals, and stable ranking behavior to support interpretation by candidates and recruiters.
- Integrate explicit user feedback and apply the Rocchio relevance-feedback method to update learned representations and recompute rankings reproducibly.
- Implement AutoFit policies that convert matching results into controlled outcomes according to consent, thresholds, interaction state, quota, cooldown, time-zone, and quiet-hour rules.
- Support expiring email-action links with hashed token storage, a non-mutating confirmation page, POST execution, action-state tracking, and audit logging.
- Evaluate the algorithmic behavior and system workflows using reproducible experiments, automated tests, scripted acceptance scenarios, and security and reliability checks appropriate to the project scope.

## 1.4 Scope of the Thesis

This thesis focuses on an academic prototype for recruitment in the information technology domain. Its functional scope includes a public job portal; authenticated candidate, recruiter, and administrator workspaces; CV and profile management; job and employer management; CV–JD matching; personalized job recommendation; application workflows; explicit relevance feedback; AutoFit policy configuration; notifications and actionable email; analytics; and audit records.

The data-processing scope includes extraction of text from supported CV documents, OCR fallback for image-based content where the configured runtime permits it, hard and soft validation, bilingual-oriented text normalization, TF-IDF vectorization, cosine-similarity scoring, score normalization, relevance labeling, and Rocchio-based feedback updates. PostgreSQL is used as the primary data store, and database evolution is managed through Flyway migrations. The implemented system follows a client–server architecture with a React and TypeScript frontend and a Spring Boot backend exposing REST APIs.

The thesis does not attempt to implement a complete enterprise applicant tracking system. Interview scheduling, offer management, payroll, and full employee lifecycle management are outside the core scope. The system does not automatically submit applications to external third-party job boards, and it does not use a large language model to make unrestricted recruitment decisions. Distributed microservices, large-scale message brokers, and semantic embedding models may be discussed as future extensions but must not be presented as implemented core components unless they are added and verified before the final report is submitted.

The evaluation scope is also bounded. Controlled or synthetic datasets may be used to demonstrate whether the Rocchio feedback mechanism changes rankings in the expected direction, but such results do not establish recruitment quality in a production population. Any scraped, seeded, or synthetic data used in experiments must be identified by provenance and purpose. Claims about usability, performance, security, or production readiness will be limited to the test environment, participants, procedures, and evidence actually available at the time of the final evaluation.

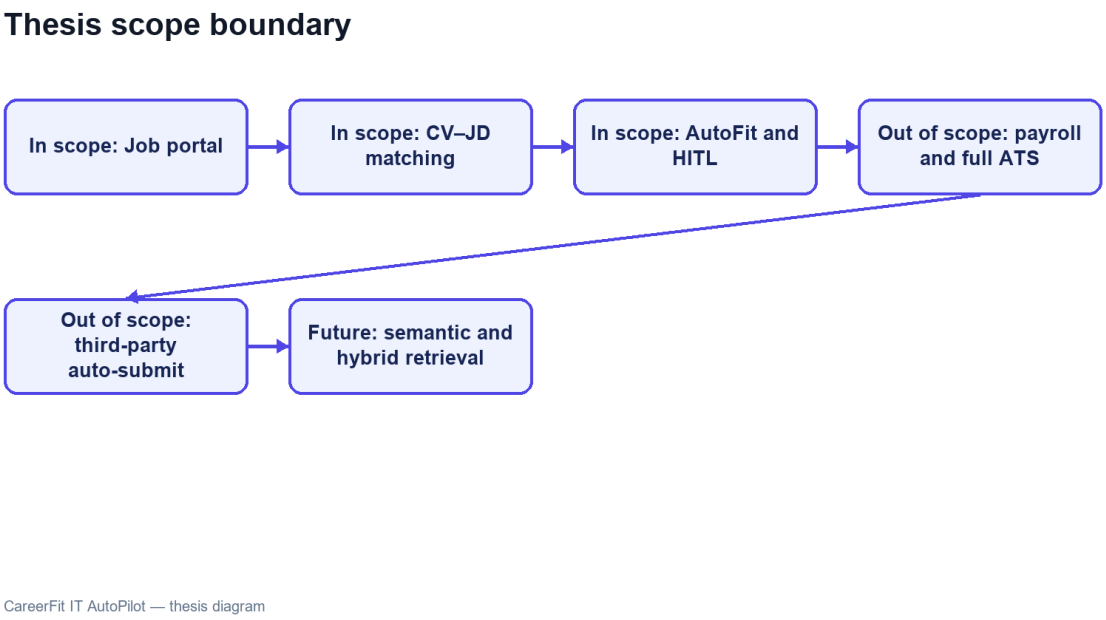



*Figure 1.2. Scope boundary of the CareerFit thesis*

## 1.5 Contributions of the Thesis

The principal contribution of this thesis is the design and implementation of an integrated, controlled recruitment workflow rather than the invention of a new machine-learning model. The contributions are summarized as follows:

- An integrated recruitment platform that combines a job portal, CV–JD matching, personalized recommendation, application management, and role-specific operational workspaces.
- A transparent text-scoring pipeline using TF-IDF and cosine similarity, with normalized results, labels, reasons, validation signals, and explicit separation between matching and recommendation objectives.
- A feedback-learning workflow based on the Rocchio method, designed to update learned representations from explicit feedback and to support reproducible recomputation rather than uncontrolled cumulative modification.
- A policy-driven AutoFit layer that separates relevance scoring from business actions and applies Human-in-the-Loop controls, user consent, operational constraints, and previous interaction states.
- An actionable email mechanism with expiring, hashed tokens, a non-mutating GET confirmation page, POST redemption, and action-state tracking.

• A traceable evaluation structure that distinguishes algorithm experiments from functional, integration, acceptance, performance, reliability, and security evidence, while explicitly documenting threats to validity.



| Thesis objective | Implemented contribution | Evaluation evidence |
|---|---|---|
| Analyze IT recruitment workflows | Role-specific requirements and Human-in-the-Loop boundaries | Use cases and traceability matrix |
| Implement CV–JD matching | TF-IDF, cosine scoring, explanations, and persisted Matchings | Algorithm benchmark and backend tests |
| Learn from feedback | Rocchio updates followed by scheduled recomputation | Baseline-versus-learned ranking metrics |
| Support controlled automation | AutoFit policy, quota, cooldown, notification, and audit controls | Security tests and P0 E2E flows |
| Deliver an integrated platform | Spring Boot, React/TypeScript, PostgreSQL, and Flyway | Build, runtime health, and browser evidence |

*Table 1.1. Mapping of objectives, contributions, and evidence*

## 1.6 Thesis Organization

The thesis contains six chapters. Chapter 1 introduces the problem, objectives, scope, and contributions. Chapter 2 presents the theoretical background and related work, including TF-IDF, cosine similarity, Rocchio feedback, ranking metrics, Human-in-the-Loop control, and relevant security foundations.

Chapter 3 covers requirements, use cases, architecture, data design, security, and deployment. Chapter 4 explains the implementation. Chapter 5 presents the evaluation method and verified results, and Chapter 6 discusses achievements, limitations, future work, and the final conclusion.

# CHAPTER 2. THEORETICAL BACKGROUND AND RELATED WORK

## 2.1 Recruitment Information and CV–Job Description Matching

Recruitment matching is an information-retrieval and decision-support problem in which heterogeneous evidence about candidates and vacancies must be compared. A curriculum vitae commonly describes education, employment history, projects, technical skills, certifications, languages, and contact information. A job description describes responsibilities, required and preferred skills, seniority, location, employment conditions, and organizational context. Some attributes are structured, such as years of experience or location, while others appear in free text and may use inconsistent terminology. Consequently, an exact database filter is useful for hard constraints but insufficient for ranking the overall relevance of a candidate and a position.

The same information supports different user tasks. From the candidate perspective, the system retrieves and ranks jobs that fit a CV or desired profile. From the recruiter perspective, it ranks applicants or discovers potential candidates for a specific vacancy. These directions share document representations and similarity functions, but they are not identical business decisions. A relevant job recommendation does not imply that an application has been submitted, and a high candidate–job similarity does not imply that a recruiter should accept the candidate. The score is therefore evidence for prioritization rather than a final employment decision.

Recruitment data also has domain-specific ambiguity. A technology may be expressed by an abbreviation, a product name, or a broader skill family; job titles vary across companies; and the same term may have different importance at junior and senior levels. Open recruitment corpora are relatively uncommon because CVs contain personal information. The Djinni Recruitment Dataset illustrates both the scale of the matching problem and the value of anonymized, documented corpora, providing more than 150,000 jobs and 230,000 candidate profiles in English and Ukrainian [6]. CareerFit is narrower: it focuses on IT recruitment and uses explicit fields and normalized text so that matching behavior can be inspected within the project scope.

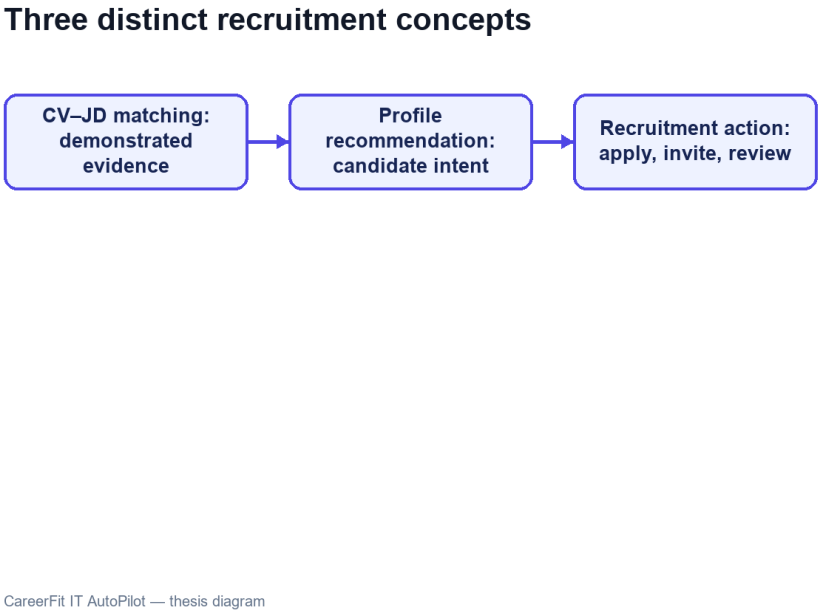



*Figure 2.1. Distinction between matching, recommendation, and recruitment action*

## 2.2 Text Representation and Preprocessing

### 2.2.1 Document Normalization

Before vectorization, text must be converted into a consistent representation. A typical pipeline extracts machine-readable text, normalizes character encoding and case, separates or standardizes tokens, removes formatting noise, and optionally filters stop words or applies stemming or lemmatization. In recruitment documents, preprocessing must preserve discriminative technical tokens such as framework names, programming languages, version-like expressions, and abbreviations. Aggressive removal can make a CV and a JD appear less related even when they share an important technology.

Bilingual data adds further complexity. Vietnamese contains diacritics and multi-syllable expressions separated by spaces, while English technical terminology frequently appears unchanged inside Vietnamese sentences. A practical pipeline must apply deterministic normalization to equivalent inputs and maintain a controlled vocabulary. It should also keep structured attributes—such as location, language, seniority, and salary mode—available for validation or filtering instead of forcing every business constraint into a single text vector.

Preprocessing quality directly affects every later score. Missing OCR text, malformed documents, extremely short descriptions, duplicated boilerplate, and

contradictory structured fields can distort term frequencies. For this reason, validation belongs before scoring. Hard validation rejects inputs that cannot support meaningful processing, while soft validation allows processing but records warnings or suggestions. This distinction prevents the ranking engine from silently assigning apparently precise scores to unusable data.

### 2.2.2 Bag-of-Words Representation

In a bag-of-words representation, a document is represented by the terms it contains and their weights; word order is not modeled directly. If the vocabulary contains |V| terms, each document is mapped to a vector in R^|V|. This representation is sparse because a particular CV or JD contains only a small portion of the complete vocabulary. Its main advantages are computational simplicity, reproducibility, and the ability to inspect which terms contribute to similarity. Its main limitation is lexical dependence: synonyms and related skills do not match unless normalization or an explicit mapping connects them.

## 2.3 TF-IDF and Cosine Similarity

### 2.3.1 Term Frequency and Inverse Document Frequency

The vector-space model represents documents and queries as weighted term vectors and ranks documents according to their geometric relationship [1]. TF-IDF is a standard weighting family for this model [2]. Term frequency (TF) reflects how strongly a term occurs in a particular document. Inverse document frequency (IDF) reduces the influence of terms that occur in many documents and increases the relative influence of terms that distinguish a smaller subset of the corpus.

Let tf(t,d) denote the frequency of term t in document d, let N be the number of documents in the reference corpus, and let df(t) be the number of documents containing t. A basic formulation is:

$$tfidf(t,d) = tf(t,d) \times log(N / df(t)).$$

Implementations often use logarithmic TF scaling, smoothed IDF, or vector normalization to handle repeated terms and unseen values. The exact formulation changes numeric scores, so a thesis must document the implemented formula rather than referring to TF-IDF as if it were a single universal function. A controlled or static reference corpus can improve score stability because the IDF of a term does not

change whenever a user adds one document; however, it may become less representative as technologies and hiring terminology evolve.

### 2.3.2 Cosine Similarity

Cosine similarity compares the direction of two vectors rather than their unnormalized magnitude. For vectors x and y, it is defined as:

$$cos(x,y) = (x \cdot y) / (||x||_2 \, ||y||_2).$$

For non-negative TF-IDF vectors, cosine similarity usually lies between 0 and 1. A larger value indicates a greater proportion of shared weighted terms. Length normalization is valuable for CV–JD comparison because a long CV should not receive a higher score merely because it contains more words. TF-IDF-weighted cosine distance has also been used as an efficient and reproducible document-alignment method [3].

Cosine similarity remains a lexical measure. If a JD uses “container orchestration” while a CV only uses “Kubernetes,” the vectors may not overlap unless the vocabulary or normalization layer establishes that relationship. Conversely, shared generic terms can increase similarity without proving that mandatory requirements are satisfied. CareerFit therefore treats similarity as one component of a broader workflow that also includes structured fields, validation, labels, reasons, user feedback, and policy conditions.

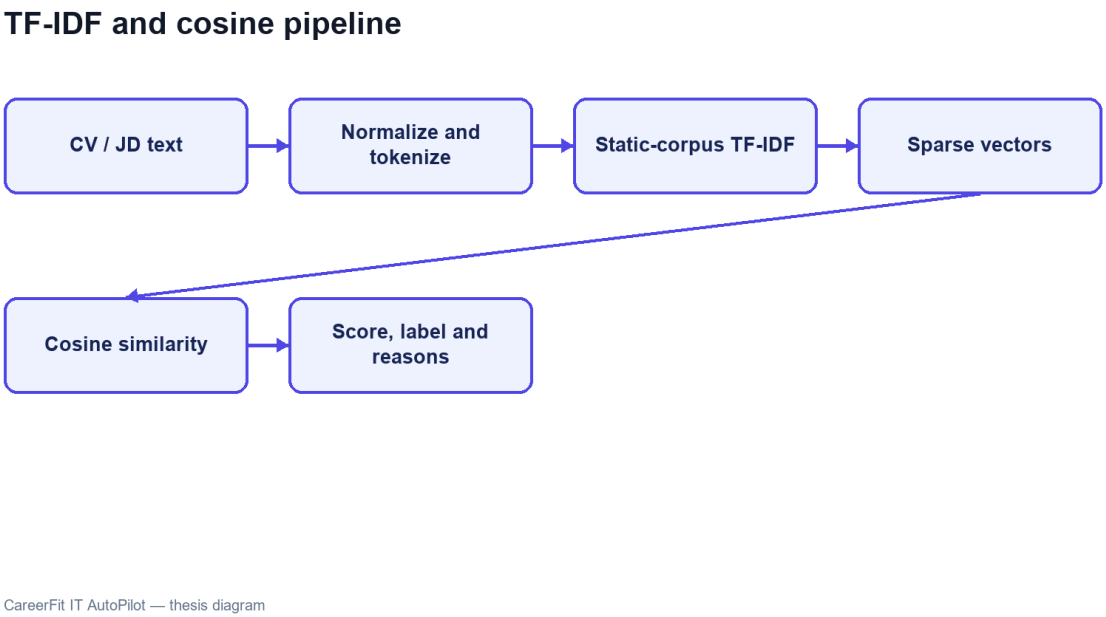



*Figure 2.2. TF-IDF vectorization and cosine-similarity pipeline*

## 2.4 Matching, Recommendation, and Ranking

Matching and recommendation can be formalized as ranking tasks. Given a query representation q and a candidate set D, the system computes a relevance score s(q,d) for each d □ D and orders the results by decreasing score. For candidate-to-job matching, q may be a CV vector and D the active job vectors. For recruiter-side discovery, q may be the selected JD and D the available candidate or CV vectors. For profile-based recommendation, q represents the candidate's desired role, skills, location, seniority, and other preferences rather than one uploaded CV.

This separation matters because a CV describes demonstrated history, whereas a preference profile describes intent. A candidate may have experience in Java but seek a position involving Go; a CV-only ranking and a preference-based recommendation can therefore produce different, legitimate lists. Storing the workflows separately also makes evaluation more defensible: the relevance labels, candidate sets, and user objectives for one task should not be assumed to be valid for the other.

A raw similarity value is not automatically a calibrated probability of hiring success. Mapping it to a percentage or label is a presentation and business-rule decision, not a statistical guarantee. Thresholds such as Low, Medium, High, or Potential should be documented, tested at boundary values, and kept distinct from application status. Stable secondary sorting is also necessary when multiple items have equal scores so that repeated requests do not appear inconsistent.

Recent resume–job matching research increasingly uses dense encoders and contrastive learning. ConFit v2, for example, improves resume–job representations through hypothetical resumes and hard-negative mining [7]. Such approaches can capture semantic relations beyond exact terms, but they require training data, model governance, and an evaluation design appropriate to the domain. CareerFit intentionally begins with a transparent lexical baseline whose calculations can be reproduced and explained, while treating semantic or hybrid retrieval as future work rather than an implemented result.

## 2.5 Relevance Feedback and the Rocchio Method

### 2.5.1 Relevance Feedback

Relevance feedback uses judgments about retrieved items to modify a query or profile representation. Instead of assuming that the original text completely expresses the user's information need, the system observes which results are marked relevant or non-relevant. Explicit feedback is especially useful in recruitment because the importance of related skills may be known to the candidate or recruiter but omitted from the original CV, profile, or JD.

The classic Rocchio method updates a query vector by combining the original query with the centroids of relevant and non-relevant document vectors [4]. Let $q_0$ be the original vector, $D_r$ the set of relevant feedback documents, and $D_{nr}$ the set of non-relevant documents. A common form is:

$$q_m = \alpha q_0 + (\beta / |D_r|) \Sigma(d \in D_r)\, d - (\gamma / |D_{nr}|) \Sigma(d \in D_{nr})\, d.$$

The parameters α, β, and γ control retention of the original representation, attraction toward relevant examples, and movement away from non-relevant examples. They are hyperparameters rather than universal constants. Positive and negative feedback may also be weighted differently because an explicit rejection can be ambiguous: a user may reject a job because of location, timing, or salary even when its technical content is relevant.

### 2.5.2 Feedback Semantics in CareerFit

CareerFit distinguishes feedback such as GOOD_MATCH, POTENTIAL, BAD_MATCH, and NOT_INTERESTED. These labels should not be collapsed without a documented mapping. GOOD_MATCH provides a strong positive relevance signal. POTENTIAL may represent partial or transferable fit rather than direct relevance. BAD_MATCH is a negative judgment about the match. NOT_INTERESTED can reflect personal intent and may be weaker evidence about technical similarity. The implementation chapter must state exactly how each event affects the learned vector and whether role or channel changes its weight.

A robust update process retains the immutable base vector and recomputes the learned vector from the relevant feedback history. Repeatedly applying an update to the previously learned vector can create cumulative drift: running the same

recomputation more than once may change the output even though no new feedback exists. Idempotent recomputation means that the same base vector, feedback set, and parameters produce the same learned vector. This property is important for retries, scheduled processing, debugging, and auditability.

Feedback learning must be evaluated causally rather than by reporting only a final score. A controlled experiment records the baseline ranking, introduces a defined feedback event, recomputes the representation, and measures the effect on separate holdout items. If the positive item used to provide feedback is also the sole item used to claim improvement, the result is circular. Chapter 5 therefore separates feedback examples from holdout evaluation and explicitly labels synthetic scenarios.

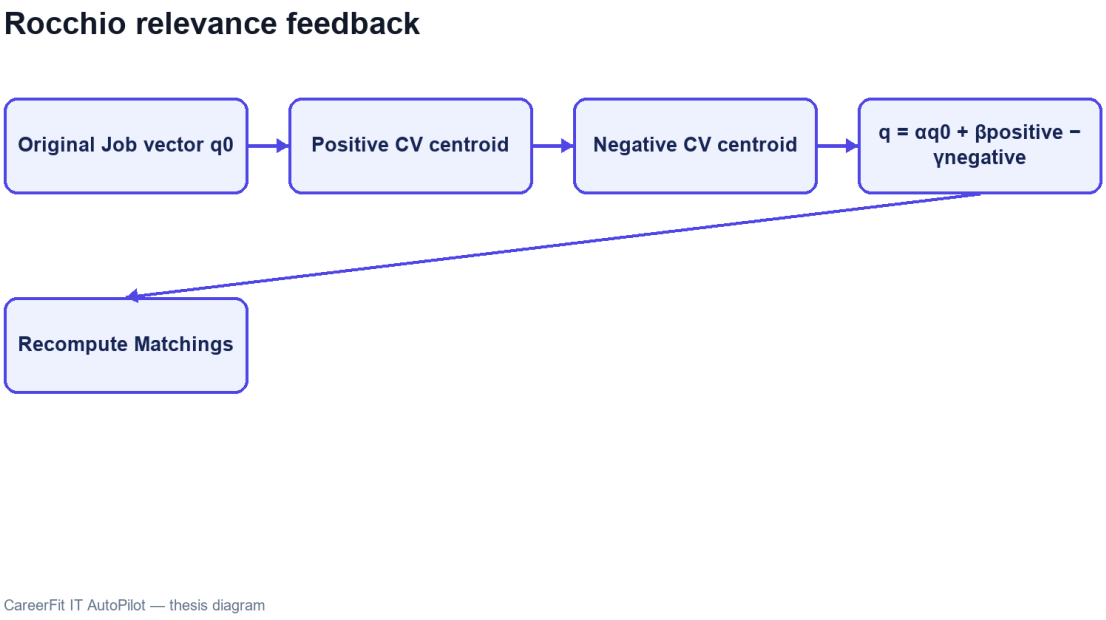



*Figure 2.3. Rocchio relevance-feedback vector update*

## 2.6 Evaluation Metrics for Ranked Results

Ranking quality cannot be adequately described by ordinary classification accuracy because users observe an ordered list and usually inspect only the first few items. Evaluation therefore requires relevance judgments and metrics that account for the cutoff position K, the location of relevant results, and, where available, graded relevance.

### 2.6.1 Precision@K, Recall@K, and HitRate@K

Precision@K is the proportion of the top K results that are relevant. Recall@K is the proportion of all known relevant items that appear in the top K. HitRate@K records whether at least one relevant item appears in the first K positions. Precision emphasizes the usefulness of the visible list, recall emphasizes coverage, and HitRate provides a simple task-success indicator. The metrics answer different questions and should not be substituted for one another.

### 2.6.2 Mean Reciprocal Rank

For a query, reciprocal rank is 1/r, where r is the rank of the first relevant result; it is zero if no relevant result is retrieved. Mean Reciprocal Rank (MRR) averages this value across queries. MRR strongly rewards placing the first relevant result near the top but ignores the ordering of additional relevant items. It is therefore appropriate when finding one useful job or candidate is the primary objective, but it should be accompanied by a broader top-K metric when multiple relevant results matter.

### 2.6.3 Discounted Cumulative Gain

Discounted Cumulative Gain (DCG) supports graded relevance and discounts useful items that occur at lower ranks. Normalized DCG divides the observed DCG by the ideal ordering for the same relevance judgments, producing nDCG values that are comparable across queries. Järvelin and Kekäläinen introduced cumulated-gain measures to credit systems for ranking highly relevant documents before less relevant ones [5]. One common formulation is:

$$DCG@K = \Sigma_{i=1..K} (2^{rel_i} - 1) / \log_2(i + 1), \quad nDCG@K = DCG@K / IDCG@K.$$

Metric values are meaningful only when the relevance labels, candidate set, cutoff K, and aggregation procedure are documented. A large improvement on a synthetic scenario demonstrates behavior under that scenario; it does not by itself demonstrate production hiring quality. Chapter 5 therefore reports dataset provenance, baseline construction, holdout logic, repeated-run behavior, and threats to validity together with the metric values.

## 2.7 Human-in-the-Loop and Explainable Automation

### 2.7.1 Levels of Human Involvement

Human-in-the-Loop is not a single interface feature. Human involvement can occur during data validation, model configuration, review of recommendations, approval of actions, exception handling, and post-action feedback. NIST describes human–AI configurations as ranging from manual decision making through systems that provide an additional opinion or defer to an expert, to more autonomous operation [9]. The appropriate configuration depends on the consequences and failure modes of the task.

Recruitment decisions can materially affect people, so a similarity engine should assist prioritization rather than act as an unquestioned authority. CareerFit separates perception, decision support, action, learning, and audit. The matching engine produces evidence; the AutoFit policy evaluates whether an action is permitted; a user may review or confirm important actions; feedback changes later ranking; and audit records support investigation. This design does not eliminate bias, but it makes the control points and system state more explicit.

### 2.7.2 Explainability and Auditability

Explainability answers why a particular result or action was produced, whereas auditability concerns whether the inputs, rules, actor, channel, time, and outcome can be reconstructed. A list of overlapping skills can provide a local explanation of lexical similarity, but it is not proof that the entire model or business decision is fair. Likewise, an audit log records what occurred but does not guarantee that the recorded policy was appropriate.

Research on algorithmic hiring warns that claims about bias mitigation and performance must be evaluated against actual practices, data, and institutional context [10]. Explanations can support understanding, but they should be grounded in the features used by the system. Work on explainable recommendation similarly distinguishes recommendation scoring from the generation of user-facing reasons and emphasizes fact-grounded explanations [14]. CareerFit therefore favors inspectable match reasons and policy outcomes over unsupported natural-language claims about candidate suitability.

The NIST AI Risk Management Framework organizes AI risk work around governance, mapping, measurement, and management [9]. CareerFit is not claimed to be a complete implementation of that framework; however, its design aligns with several practical principles: document scope and limitations, preserve human authority for consequential actions, measure behavior using defined evidence, and maintain records that support review.

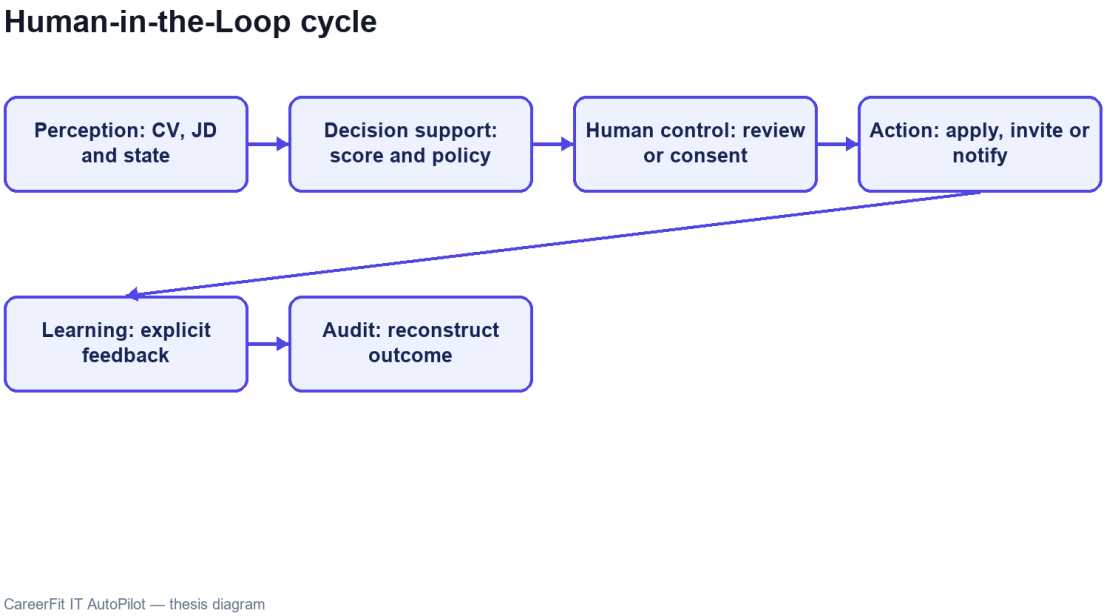



*Figure 2.4. Human-in-the-Loop control cycle in CareerFit*

## 2.8 Web Architecture, Security, and Audit Foundations

### 2.8.1 Client–Server and REST

CareerFit uses a web client and a server-side application connected through HTTP APIs. The client is responsible for presentation, navigation, local interaction state, and invoking authorized operations. The backend enforces authentication, authorization, validation, business rules, transactions, persistence, and background processing. This separation prevents browser-side presentation logic from becoming the authority for protected actions.

REST is an architectural style characterized by constraints including client–server separation, stateless interaction, cacheability where appropriate, a uniform interface, layered components, and optional code-on-demand [11]. A REST API models resources and transfers representations of their state. In practice, merely using JSON over HTTP does not guarantee that every REST constraint is satisfied; Chapter

4 therefore describes CareerFit as a REST-oriented API and documents its actual endpoint and state-transition behavior.

### 2.8.2 Authentication, Authorization, and Action Tokens

Authentication establishes the identity associated with a request, while authorization determines whether that identity may perform an operation. Role-Based Access Control assigns permissions according to roles such as Candidate, Recruiter, or Administrator, but ownership checks are still required; for example, one candidate must not access another candidate's private CV merely because both share the same role.

JSON Web Token is a compact, URL-safe format for transferring claims that can be signed or message-authentication-code protected and may include registered claims such as subject, audience, expiration time, issued-at time, and token identifier [12]. A valid signature alone is not sufficient: the application must validate the expected algorithm and relevant claims and must apply its own account and authorization rules. Short-lived access tokens and purpose-specific action tokens address different workflows and should not be treated as interchangeable credentials.

An actionable email link can be opened by the intended user, forwarded accidentally, or visited automatically by a mail-security scanner. A safer target pattern lets a GET request display a confirmation page without changing business state and requires a deliberate POST request to execute the action; the current CareerFit email-action implementation does not yet satisfy this target. The action token should be random or cryptographically protected, purpose-bound, expiring, and single-use. Server-side consumption state or an equivalent replay-prevention mechanism is needed because expiration alone does not prevent the same valid link from being used repeatedly.

### 2.8.3 Audit Logging

Audit logging records security-relevant and business-relevant events so that operations can be monitored and investigated. NIST guidance emphasizes a managed logging process that includes generation, transmission, storage, access, analysis, and disposal rather than treating logs as incidental text files [13]. For recruitment automation, useful fields include the actor, action, target, source channel, relevant policy or score snapshot, result, and timestamp. Sensitive CV content and credentials should not be copied indiscriminately into logs.

An append-oriented audit history supports traceability, but database permissions, retention, integrity, and access control determine whether that history is trustworthy. Operational logs used for debugging and domain audit records used to explain a business action may have different schemas and retention requirements. Chapter 3 defines these responsibilities at the design level, and Chapter 4 describes the mechanisms actually implemented in CareerFit.

## 2.9 Related Work and Research Gap

Prior work can be grouped into lexical retrieval, learned resume–job representation, explainable recommendation, and human-centered governance. The classical vector-space and relevance-feedback literature provides transparent mathematical foundations [1], [4]. Contemporary resume–job matching methods such as ConFit v2 use trained dense representations and hard-negative strategies to improve semantic matching [7]. Real-world observational research comparing human and LLM resume ratings found only minor correlation between the two, indicating that automated and human judgments should not be treated as interchangeable [8]. Explainable recommendation research seeks to accompany scores with understandable, preferably grounded reasons [14].

CareerFit does not attempt to outperform these research systems on a shared benchmark. Its engineering gap is different: connecting an inspectable lexical baseline and explicit relevance feedback to an end-to-end recruitment workflow with role-based interfaces, policy gating, actionable email, and auditable state changes. The resulting contribution is system integration and controlled automation within an IT-focused academic prototype.

*Table 2.1. Positioning of CareerFit against major approach categories*

| Approach | Strength | Limitation relative to CareerFit scope | CareerFit position |
|---|---|---|---|
| Lexical TF-IDF/cosine | Transparent, efficient, reproducible | Limited semantic equivalence | Implemented baseline with normalization, reasons, and validation |
| Dense resume–job encoders | Captures semantic relations | Requires training data, governance, and benchmark validation | Future hybrid/semantic extension; not claimed as implemented |
| General job portal | Strong search, publication, and application UX | May not expose scoring, feedback, or policy internals | Combines portal workflows with inspectable matching and automation |
| Fully automated decision pipeline | Reduces manual steps | Higher control, accountability, and error-propagation risk | Policy-gated HITL actions with confirmation and audit |
| Explainable recommender | Provides user-facing reasons | Explanation may be unfaithful if not grounded | Uses reasons tied to processed fields and lexical evidence |

The comparison shows that CareerFit deliberately accepts the semantic limitations of a lexical baseline in exchange for inspectability and implementation control. Its research value must therefore be assessed through reproducible behavior, workflow integration, and clearly stated limitations rather than through unsupported claims of general AI superiority.

## 2.10 Chapter Summary

This chapter presented the main ideas used by CareerFit. TF-IDF and cosine similarity provide an inspectable lexical baseline, while Rocchio uses explicit feedback to adjust ranking. Ranking metrics measure ordered results, and Human-in-the-Loop policies keep similarity evidence separate from recruitment actions. Chapter 3 turns these ideas into system requirements and design decisions.

# CHAPTER 3. SYSTEM ANALYSIS AND DESIGN

## 3.1 System Analysis Context

CareerFit is designed as a role-based system for IT recruitment. Its boundary includes the React frontend, Spring Boot backend, PostgreSQL database, CV storage, background scheduler, and optional email provider. External job boards, interview scheduling, offers, payroll, and unrestricted automated hiring remain outside the implemented scope.

The central design distinction is between evidence, business state, and action. A matching record contains evidence about the similarity between one CV and one job. An application record represents a candidate's participation in a recruitment process. An automation policy represents consent and operational preferences. These concepts must remain separate: a high similarity score must not silently become an application, and an application status must not be interpreted as a relevance label.

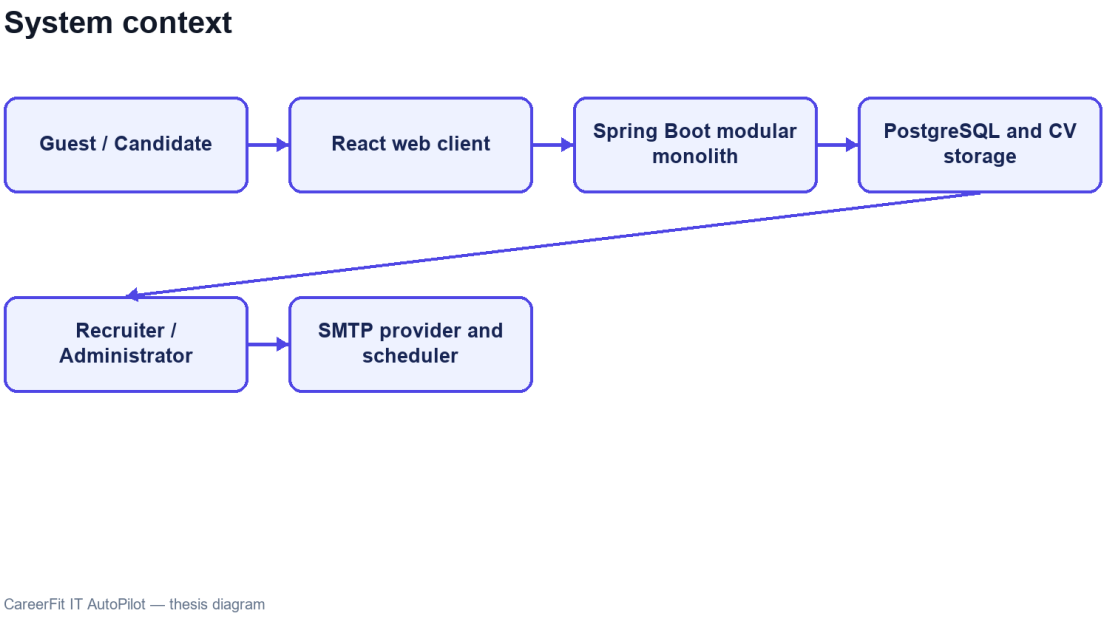



*Figure 3.1. CareerFit system context*

## 3.2 Stakeholders and Actors

CareerFit serves four interactive roles and two supporting actors. Guest is a frontend state rather than a persisted account role. Candidate, Recruiter, and Administrator are persisted roles in user_account. Background processing and the mail provider are non-human actors that execute scheduled work and deliver messages.

*Table 3.1. Actors and responsibilities*

| Actor | Primary responsibilities | Access boundary |
|---|---|---|
| Guest | Browse public jobs, search suggestions, job details, featured employers, and public market analytics | Public GET operations only; authentication is required for personalized data or application actions |
| Candidate | Maintain profile, CVs, and portfolio; view matching and recommendation results; apply or withdraw; provide feedback; configure personal automation | Candidate-owned resources and candidate APIs |
| Recruiter | Maintain employer profile and jobs; inspect applicants and discovered candidates; invite candidates; update application status; view recruiter analytics | Recruiter APIs and jobs owned by the authenticated recruiter |
| Administrator | Monitor system summaries, users, jobs, email actions/tokens, and audit records; moderate jobs and trigger controlled maintenance operations | Administrative APIs only |
| Background scheduler | Recompute stale matches, send digests and notifications, clean expired email actions, and execute auto-apply scans | Internal service calls under system control |
| Mail provider | Deliver passwordless or notification email generated by the backend | SMTP boundary; it does not own CareerFit business state |

Stakeholder needs sometimes conflict. Candidates want relevant opportunities, privacy, and control over automatic applications. Recruiters want efficient prioritization without losing access to applicants who do not rank highly. Administrators need visibility and recoverability without unrestricted exposure of CV content. The design therefore combines role authorization, ownership checks, policy settings, validation, explicit statuses, and audit records rather than relying on a single ranking score.

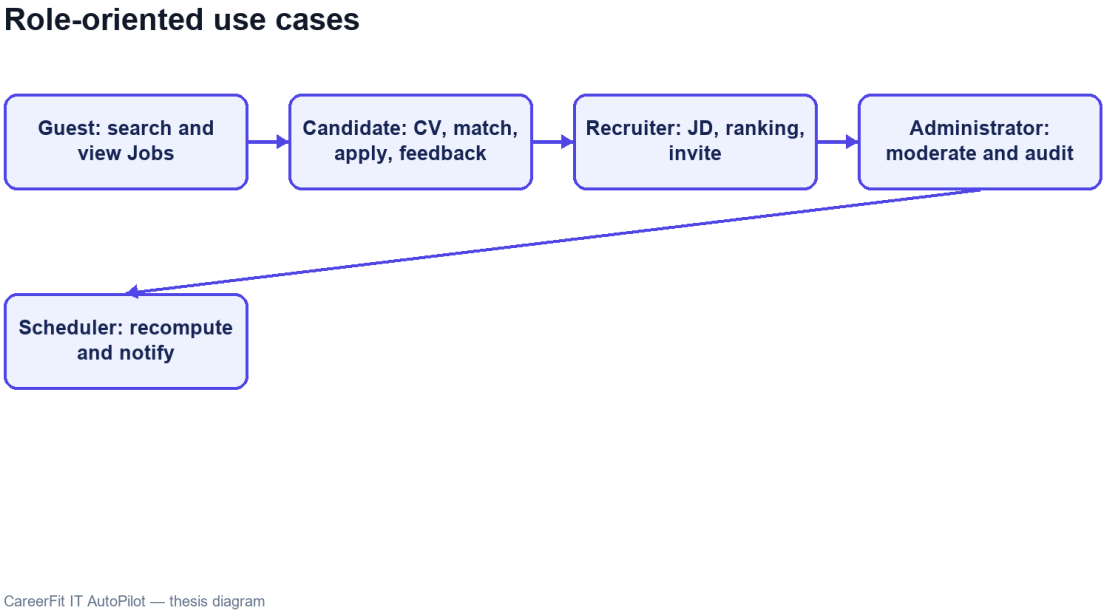



*Figure 3.2. CareerFit use-case overview*

## 3.3 Functional Requirements

The detailed Software Requirements Specification contains individual requirement identifiers. For the thesis, they are consolidated into functional groups that map to implemented modules and observable workflows.

*Table 3.2. Consolidated functional requirements*

| Group | Required capabilities | Main actor |
|---|---|---|
| Authentication and account | Registration, password login, passwordless verification, current-account lookup, role enforcement, account settings | Candidate, Recruiter, Administrator |
| Public job portal | Search, suggestions, filters, job details, featured employers, employer details, and public analytics | Guest and authenticated users |
| Candidate profile and CV | Profile and portfolio maintenance, multiple CVs, default CV selection, document upload, manual creation, status tracking, validation, extraction, and OCR fallback | Candidate |
| Job and employer management | Employer profile management; job creation, update, status change, deletion, search, and CSV export | Recruiter |
| Matching and recommendation | CV–JD scoring, labels, reasons, candidate match cards, job ranking, profile-based recommendations, and similar jobs | Candidate and Recruiter |

| Group | Required capabilities | Main actor |
|---|---|---|
| Application workflow | Manual apply, candidate application history, withdrawal, recruiter applicant view, invitation, and status update | Candidate and Recruiter |
| Feedback learning | Record explicit feedback from web or email, update learned representations with Rocchio, and mark affected matches for recomputation | Candidate |
| Automation and notification | Policy retrieval/update, pause/resume, run-now, scheduled auto-apply, digest, high-match email, quota, cooldown, and delivery logging | Authenticated user and Scheduler |
| Analytics and administration | Market, candidate, recruiter, and system summaries; event tracking; user/job moderation; audit and email-token monitoring | Candidate, Recruiter, Administrator |

The implementation exposes these groups through REST-oriented endpoints. Examples include /api/jobs/search, /api/cv/upload, /api/matches/me/cards, /api/recommendations/jobs, /api/applications, /api/recruiter/jobs/{jobId}/candidates, /api/automation/policy, and /api/admin/audit-logs. Chapter 4 describes endpoint behavior in detail. This chapter uses the endpoint groups only to establish component boundaries and traceability.

## 3.4 Non-Functional Requirements

Non-functional requirements constrain how the workflows must operate. They are expressed as design targets; Chapter 5 must distinguish targets from properties verified by measurement.

*Table 3.3. Non-functional requirements and design responses*

| Quality attribute | Requirement | Design response |
|---|---|---|
| Security | Protected resources must require authenticated and authorized access | Stateless Spring Security filter chain, JWT processing, role rules, ownership checks, BCrypt password hashing, CORS allow-list, CSP, and HSTS configuration |
| Privacy | CV and account data must not be public or unnecessarily logged | Candidate-scoped APIs, local protected storage boundary, selective DTOs, and audit metadata rather than full document content |

| Quality attribute | Requirement | Design response |
|---|---|---|
| Reliability | Repeated or failed operations should not create inconsistent domain state | Database constraints, transactions, unique keys, optimistic version columns, idempotency checks, explicit statuses, and retry-aware background work |
| Performance | Public queries and ranking views should remain responsive for the academic dataset | Database indexes, pagination, stored vectors, background scoring, and bounded result sets |
| Maintainability | Domain changes should remain localized and database evolution reproducible | Modular package structure, controller–service–repository separation, DTOs, configuration properties, Flyway migrations, and automated tests |
| Usability | Users must understand loading, errors, empty results, score context, and available actions | Role-specific pages, normalized labels and reasons, validation signals, and explicit action controls |
| Auditability | Significant automated or administrative events should be reconstructable | audit_log, notification delivery records, email-action state, timestamps, actor/source metadata, and administrative views |
| Explainability | The system should expose evidence without presenting a score as a hiring probability | Lexical term/skill reasons, separate labels and application states, policy settings, and documented limitations |

Security and auditability are defense-in-depth properties. A role rule at the URL layer does not replace ownership verification in a service, and an audit row does not make an unsafe action safe. Similarly, a database index is a performance design decision, not proof of latency under load. The evaluation chapter must test each critical claim in the environment in which it is reported.

## 3.5 Use-Case Analysis

### 3.5.1 Candidate Uploads a CV and Receives Matches

Preconditions are an active Candidate account and an authenticated request. The candidate uploads a supported document or submits manual CV data. The backend validates file type, size, and content; stores the source; extracts text directly or invokes OCR fallback; normalizes text; derives terms and skills; creates or updates the CV vector; and scores the CV against eligible active jobs. The CV status moves through explicit processing states and ends in SCORING_DONE or FAILED. Successful scores are persisted as unique CV–job matching records and returned as ordered match cards.

Alternative flows include unsupported or oversized files, insufficient extracted text, OCR failure, invalid form fields, no active jobs, and partial quality warnings. Hard errors stop scoring and preserve a failure reason. Soft warnings allow the workflow to continue but should remain visible to the user. Selecting a new default CV changes which CV supplies personalized matching and automation input; the database enforces at most one default CV per candidate.

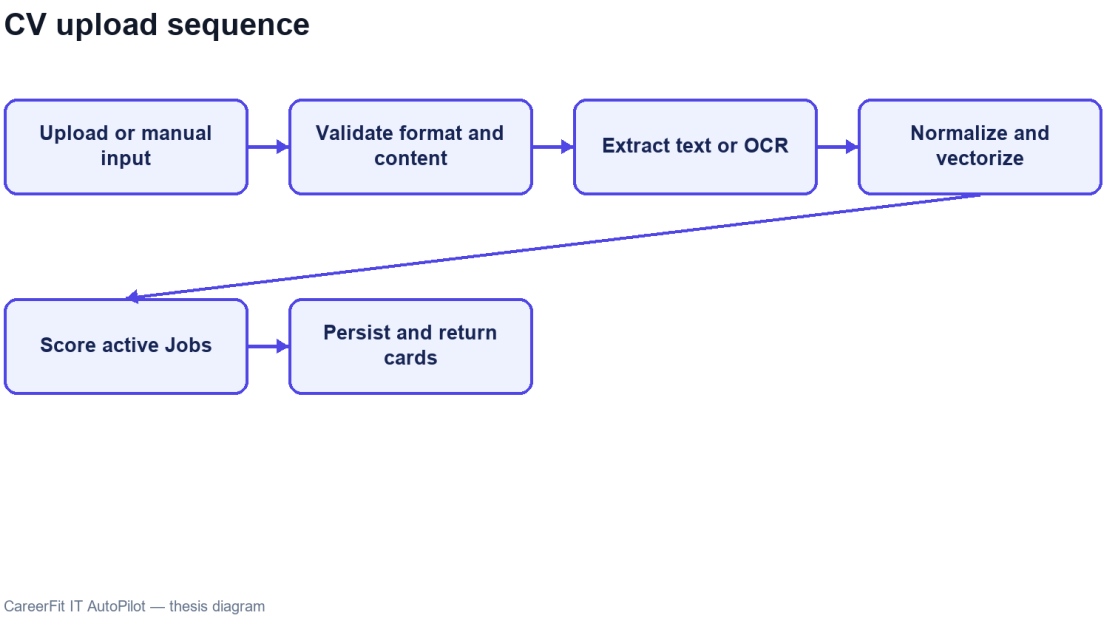



*Figure 3.3. CV ingestion and matching sequence*

### 3.5.2 Candidate Searches and Applies for a Job

A Guest or Candidate can search public jobs and open job details. Personalized scores and application actions require a Candidate account. When the candidate applies, the service resolves the authenticated candidate, verifies the job and selected/default CV, prevents a duplicate candidate–job application, and creates an application with the appropriate status and source. The candidate can list owned applications and withdraw where the current state permits it. The unique database constraint on candidate and job provides a final duplicate-prevention boundary.

### 3.5.3 Recruiter Creates a Job and Reviews Candidates

An authenticated Recruiter creates a JD with structured fields and free text. Validation checks required content and conditional salary rules. The backend normalizes and vectorizes the job, persists ownership through recruiter_id, and makes eligible jobs available for matching. Recruiter ranking and discovery endpoints combine matching evidence with whether a candidate has applied. The recruiter can filter candidates, invite a suitable candidate, or update an existing application's status. Service-level ownership checks must ensure that a recruiter cannot manage another recruiter's vacancy by changing an identifier.

### 3.5.4 User Feedback Changes Later Ranking

A Candidate submits feedback for a Matching that belongs to the Candidate's CV. The endpoint rejects another role or an unrelated Matching, stores one current judgment per actor and Matching, and starts Rocchio learning only after the feedback transaction commits. Affected Matching rows are marked needs_recompute. The scheduler then rescans those rows and clears the marker after successful scoring. Recruiter decisions use the separate application and invitation workflow rather than this Candidate feedback endpoint.

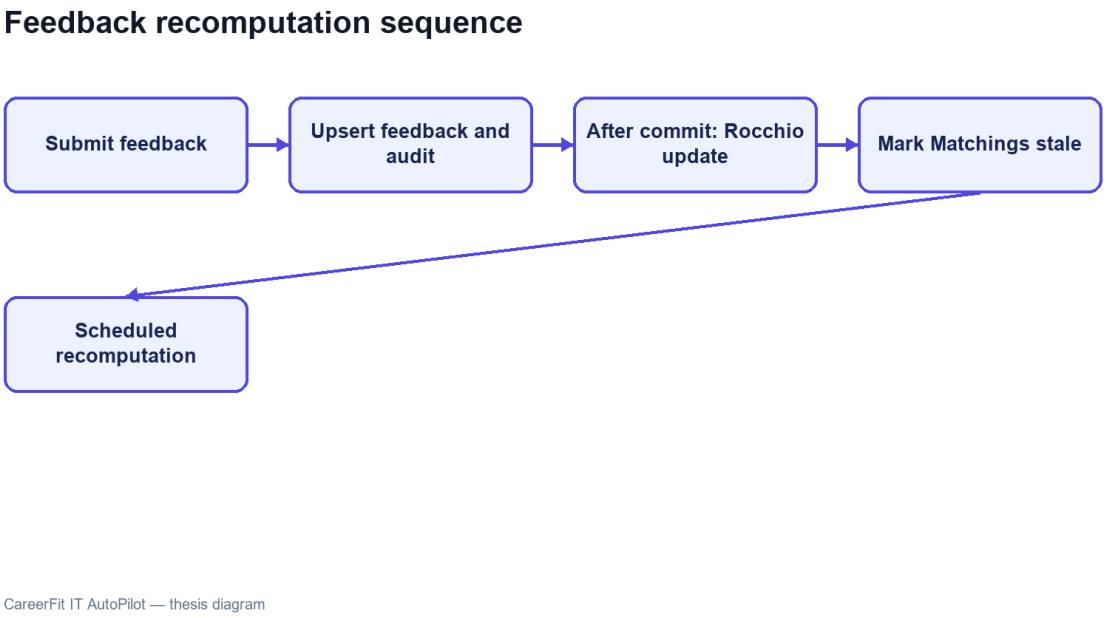



*Figure 3.4. Feedback learning and recomputation sequence*

### 3.5.5 AutoFit Executes a Policy-Governed Action

Automation begins with a persisted policy associated with a user. Candidate-oriented policy data includes enablement, score thresholds, digest and high-match notification preferences, daily limits, cooldown, quiet hours, timezone, pause state, and auto-apply options. Scheduled scans resolve the user's Candidate and default CV, require a successfully scored CV, select eligible matches, and apply policy and state checks before sending a notification or creating an application. run-now provides a controlled testing path without changing the policy semantics.

The scheduler contains five concrete tasks: stale Matching recomputation every 30 minutes; daily digest at 08:00 Asia/Ho_Chi_Minh; expired email-action cleanup at 03:00; high-match notification scans every four hours with an additional 07:00–22:00 ICT window; and auto-apply scans every two hours. The annotation expressions read their delays, cron expressions, and time zone from app.scheduler.* properties, providing one environment-configurable scheduling source.

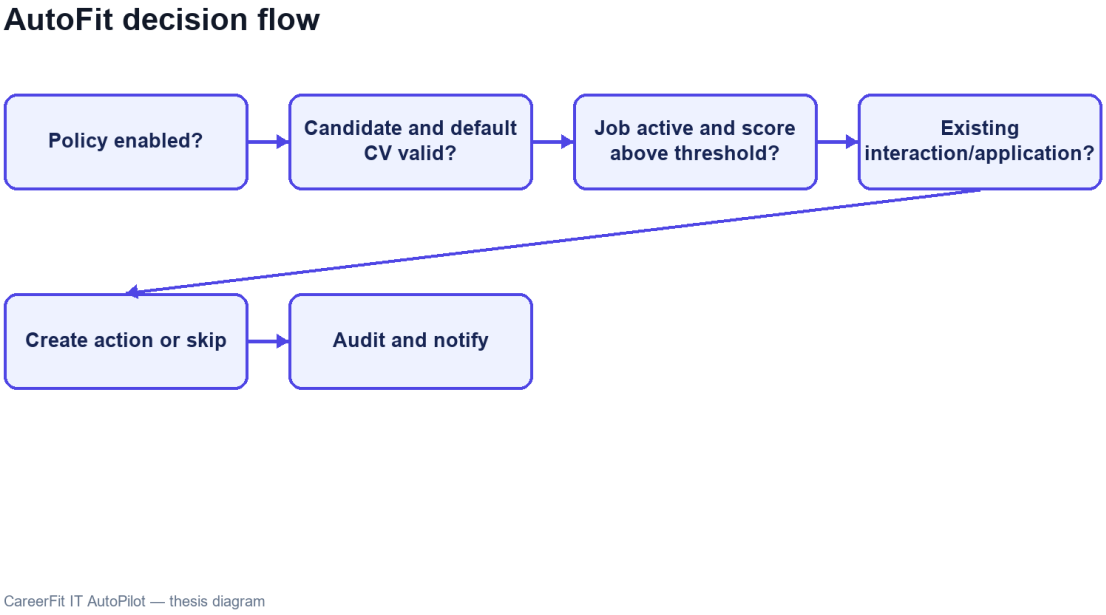



*Figure 3.5. AutoFit decision flow*

### 3.5.6 Email Feedback Action

For match notifications and digests, the backend creates EmailAction records with a high-entropy raw token for the recipient, a persisted SHA-256 token hash, and a 72-hour expiry. The public /api/email-action/redeem endpoint validates the hash, pending state, and expiry. Reuse is rejected, expired pending actions are marked expired, and a scheduled task later removes old records.

The email-action design uses two steps. GET hashes and validates the supplied high-entropy token and displays a confirmation page without changing recruitment state. A deliberate POST repeats validation, executes the supported action, marks the token as redeemed, and records the outcome. Only the SHA-256 token hash is persisted, while expiry and status checks prevent normal replay. Automated link scanners therefore cannot execute an action by merely visiting the GET URL.

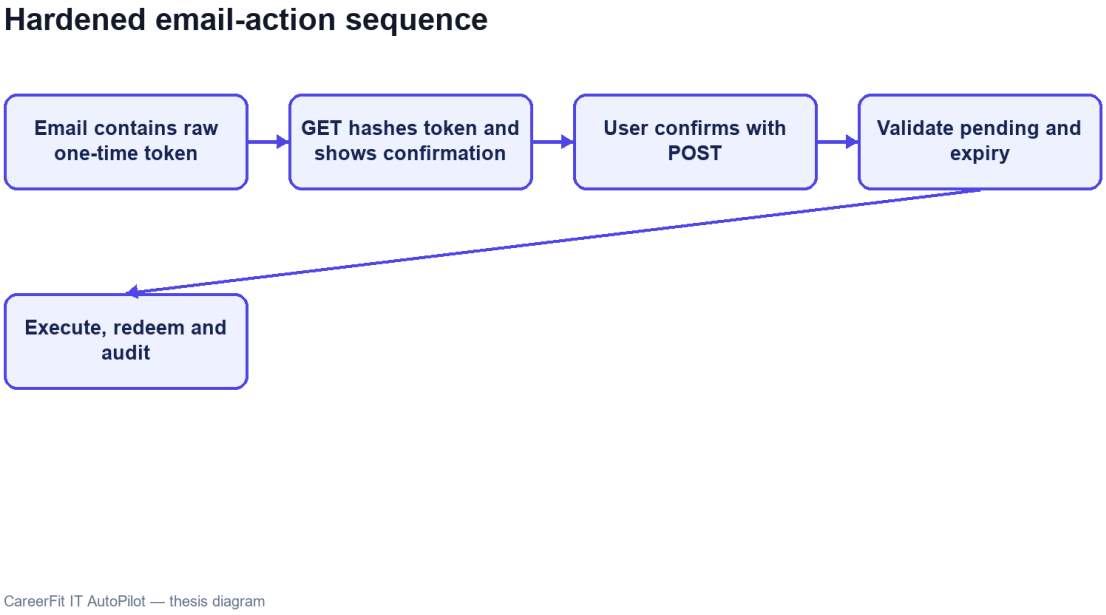



*Figure 3.6. Secure email-action confirmation and redemption sequence*

## 3.6 System Architecture

CareerFit is implemented as a modular monolith. The frontend is a separate React application, while the backend is one Spring Boot deployable application organized into domain packages. This is not a microservice architecture: modules share one process, one primary PostgreSQL database, common security and exception handling, and direct in-process service calls. The modular monolith reduces deployment and distributed-transaction complexity while preserving boundaries that can support later extraction if scale or ownership requires it.

The presentation layer contains React pages, reusable components, route guards, API mapping, and query state. It calls the backend over HTTP and must not be trusted to enforce protected business rules. The API layer contains controllers and DTO validation. The service layer owns use-case orchestration, transactions, scoring, policy decisions, and ownership checks. Repositories map domain entities to PostgreSQL through Spring Data JPA. Cross-cutting configuration covers security, CORS, async execution, exception responses, scheduling, API documentation, and health/metrics endpoints.

The backend stores relational business state in PostgreSQL. JSONB is used for naturally variable collections or snapshots such as skills, extracted terms, vectors, reasons, benefits, policy metadata, and analytic distributions. CV binaries are stored through a storage service in a configured local path or Docker volume; the database

stores file metadata and the processing result. SMTP is optional, and a no-operation mail implementation allows local workflows when outbound email is disabled.

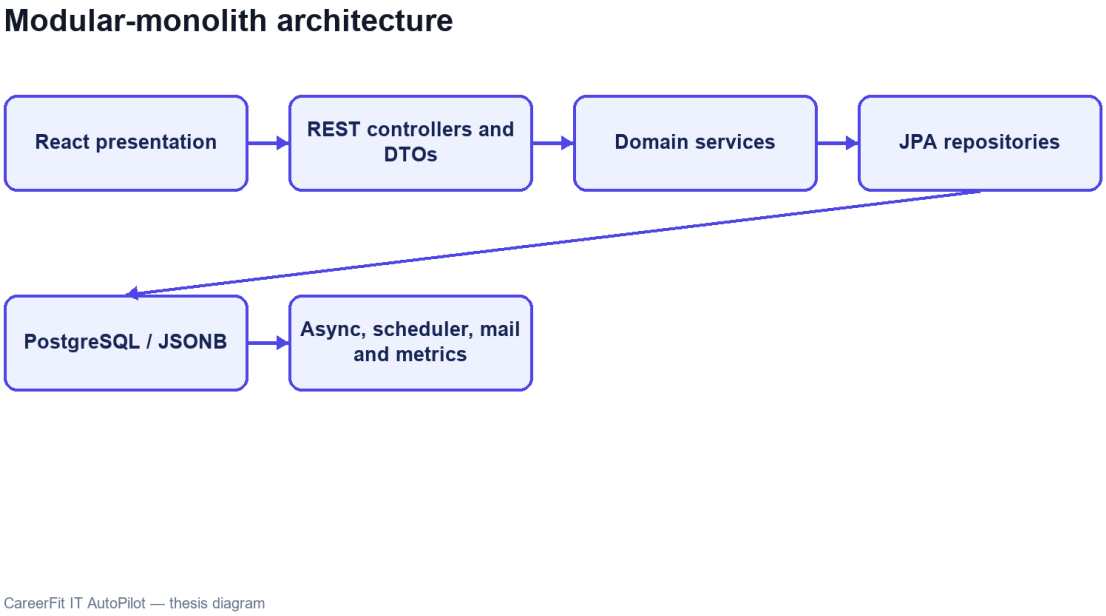



*Figure 3.7. CareerFit container and component architecture*

## 3.7 Module Design

*Table 3.4. Backend modules and responsibilities*

| Module | Main responsibility | Representative services |
|---|---|---|
| Auth and Security | Registration, login, passwordless flow, JWT validation, account resolution, role enforcement | AuthService, JwtService, security filters |
| Candidate and CV | Candidate profile, portfolio, CV lifecycle, storage, extraction/OCR, validation | CandidateProfileService, CvManagementService, CvIngestionService, PdfExtractionService |
| Job and Employer | Public job search, recruiter-owned job lifecycle, employer pages and profile | JobService, EmployerService |
| Matching | TF-IDF/cosine scoring, labels, reasons, batch matching, candidate cards and recruiter ranking | TfIdfService, ScoringService, MatchingService, MatchingBatchService, MatchingQueryService |
| Recommendation | Candidate-profile job ordering and similar-job retrieval | RecommendationService |
| Application | Manual applications, withdrawal, applicants, invitation, recruiter status transitions | ApplicationService |
| Feedback | Feedback validation, persistence, Rocchio update, recomputation signaling | FeedbackService, RocchioService |

| Module | Main responsibility | Representative services |
|---|---|---|
| Automation | Policy persistence, run-now, auto-apply, pause/resume, scheduled orchestration | AutomationPolicyService, AutoApplyService, AutomationScheduler |
| Notification | Policy guard, delivery log, SMTP/no-op sending, actionable email | NotificationPolicyGuard, NotificationEmailService, EmailActionService |
| Analytics and Admin | Market/role analytics, events, administrative dashboards, moderation and monitoring | AnalyticsService, AdvancedAnalyticsService, administrative services |
| Audit and Common | Audit persistence, shared responses, exceptions, validation utilities | Audit repository/service and common components |

The modules cooperate through service calls rather than direct controller-to-repository shortcuts for core workflows. Transaction boundaries belong at service operations that change a consistent group of records. Read-only queries may use dedicated query services or projections to avoid loading unrelated state. DTOs prevent persistence entities from becoming an accidental public API contract.

## 3.8 Data Design

### 3.8.1 Core Identity and Recruitment Data

user_account stores identity, role, activation, verification, language, and password hash. A Candidate account owns one candidate profile and multiple cv, portfolio link, and portfolio project records. A Recruiter account owns an employer_profile and jobs through recruiter_id. job stores the original JD, structured recruitment attributes, salary mode, language, status, source provenance, and vector data.

matching is the unique association between one CV and one job. It stores raw and normalized scores, label, potential metadata, reasons, and the recomputation marker. application links Candidate and Job and may reference the CV and Matching used at application time. feedback links a Matching with its actor and source channel. recommendation_interaction records behavioral events such as viewed, skipped, applied, saved, not interested, or show similar.

### 3.8.2 Automation, Communication, and Operational Data

automation_policy stores one policy per user. email_action and email_token currently represent two distinct tokenized workflows. Notification delivery records support deduplication and quota/cooldown decisions. audit_log stores actor, action, target, result, source channel, request metadata, and structured metadata. Job trend, market snapshot, and analytics event tables support aggregate dashboards without changing the transactional meaning of matching records.

*Table 3.5. Key integrity constraints*

| Constraint | Purpose |
|---|---|
| Unique normalized user email | Prevent duplicate identities that differ only by case |
| One Candidate per user and one policy per user | Preserve one-to-one domain ownership |
| Partial unique default CV per Candidate | Ensure at most one active default CV |
| Unique Matching per CV and Job | Prevent duplicate scores for the same pair |
| Unique Application per Candidate and Job | Prevent duplicate applications |
| Unique feedback per Matching and actor | Prevent ambiguous duplicate current judgments |
| Check constraints on roles, statuses, labels, salary mode, and source channels | Reject invalid domain states at the database boundary |
| Optimistic version columns on mutable core entities | Detect conflicting concurrent updates |
| Source hash index on imported jobs | Support idempotent scraped-job upsert |

Flyway migrations are the authoritative schema history. Hibernate uses ddl-auto=validate, so the application validates entity-to-schema compatibility instead of silently generating a different database. Indexes support common access paths such as active jobs, recruiter jobs, CV ownership, matching by score, applications by candidate or job, feedback history, tokens, and audit chronology.

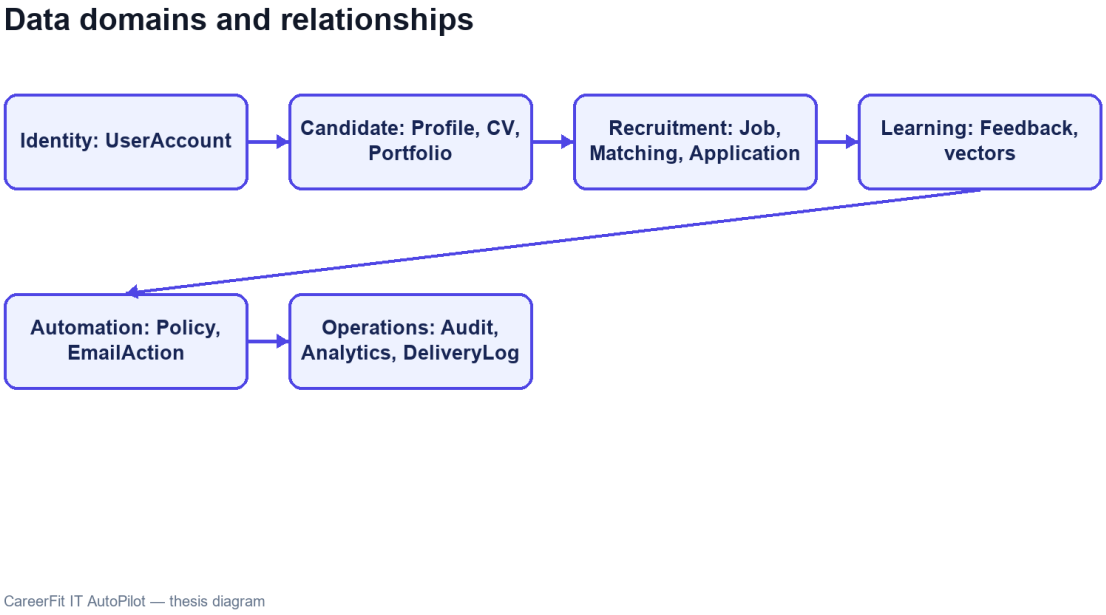



*Figure 3.8. CareerFit logical entity relationships*

## 3.9 Security, Failure, and Consistency Design

### 3.9.1 Authentication and Authorization

The backend is stateless at the HTTP session layer. A JWT authentication filter runs before username/password authentication processing, and a user-ID resolution filter runs after JWT validation. Public access is explicitly limited to authentication entry points, selected job/employer/analytics GET endpoints, health/metrics/API documentation, and email-action redemption. Candidate, Recruiter, and Administrator endpoint groups require corresponding roles. Automation requires authentication, with service logic responsible for policy ownership.

The security configuration disables CSRF because the application uses stateless bearer authentication, configures a CORS origin allow-list, denies framing, applies a content-security policy, and configures HSTS. These headers do not replace TLS termination in the deployed environment. Public Swagger and Prometheus access are acceptable for local development but should be restricted in a production deployment.

### 3.9.2 Failure Handling

CV processing represents long-running work with statuses rather than holding a browser request open until every score completes. Extraction or validation failure records a failure state and reason. CV and Job matching tasks are submitted through an after-commit executor so background work cannot read uncommitted rows. Batch scoring isolates individual pair failures, and scheduled recomputation keeps unsuccessful rows marked for later review instead of falsely marking them complete.

Email can operate through SMTP or a no-op implementation. Notification delivery outcomes are recorded as sent, skipped, or failed, and policy guards enforce deduplication, quota, cooldown, and timing rules. Email failure must not roll back an already valid matching result. Conversely, application creation and status changes use transactions and uniqueness constraints because partial persistence would change recruitment state.

### 3.9.3 Identified Design Risks

*Table 3.6. Design risks and required treatment*

| Risk | Current control | Remaining treatment |
|---|---|---|
| Unauthorized cross-account access | URL role rules and service ownership checks | Maintain negative integration tests for every identifier-based operation |
| Duplicate applications or matchings | Service checks and unique constraints | Convert database conflicts into stable API errors |
| Replayed email action | Pending/redeemed state and expiry | Implemented hashed storage, GET confirmation, and POST execution; retain replay and expiry tests |
| Link scanner triggers action | GET is non-mutating and displays confirmation | Add rate limiting and deployment-specific origin controls |
| Scheduler/configuration drift | Scheduler timing reads from app.scheduler properties | Maintain configuration and schedule-boundary tests |
| Sensitive data in logs | Structured audit metadata and DTO boundaries | Review retention, redaction, access, and exported evidence |

| Risk | Current control | Remaining treatment |
|---|---|---|
| Lexical scoring bias or omission | Reasons, validation, feedback, and HITL | Evaluate representative data; never treat score as hiring probability |
| Local file-storage loss or exposure | Configured path and Docker volume | Define backup, encryption, malware scanning, and retention before production |

## 3.10 Deployment Architecture

The development deployment uses Docker Compose for PostgreSQL and optionally the backend. PostgreSQL 16 exposes container port 5432 as host port 5433 by default. A backend process running directly on the host connects to localhost:5433; the backend container connects through the Compose network to postgres:5432. Confusing these two addresses is a common configuration failure.

The optional backend container exposes port 8080, depends on PostgreSQL health, and mounts a named volume for CV storage. The React/Vite frontend is normally run separately on port 5173 during development and calls the backend API. Environment variables override database credentials, JWT secret, application base URL, CORS origins, mail settings, OCR executable and languages, and storage path. Tesseract with Vietnamese and English language data is included in the backend container path described by the project Docker configuration.

Actuator exposes health and Prometheus metrics according to the current security configuration. PostgreSQL health checks gate backend container startup. This architecture is appropriate for reproducible local development and demonstration, but it is not by itself a production topology. A production design would additionally require managed secrets, TLS termination, restricted management endpoints, database backup and recovery, protected object storage, centralized logs, monitoring alerts, and explicit scaling and retention policies.

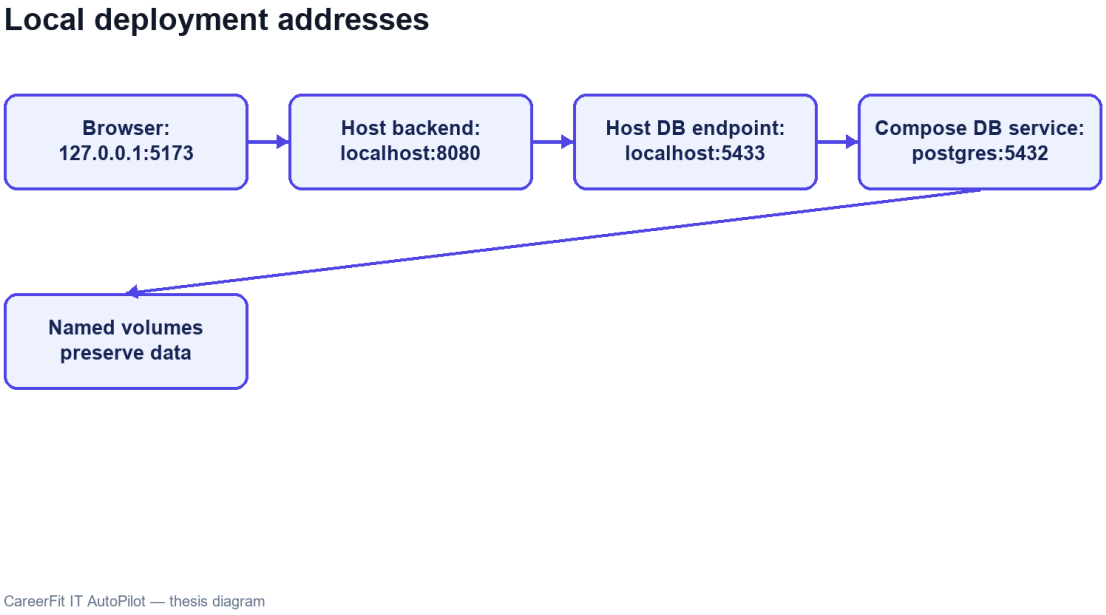



*Figure 3.9. Local and containerized deployment topology*

## 3.11 Chapter Summary

This chapter defined the actors, requirements, use cases, architecture, data model, security boundaries, failure handling, and deployment design. The main design rule is to keep matching evidence, application state, user policy, and automated actions separate. Chapter 4 explains how these decisions are implemented.

# CHAPTER 4. SYSTEM IMPLEMENTATION

## 4.1 Implementation Overview

CareerFit is implemented as two application projects and a shared runtime environment. The backend is a Java 21 application based on Spring Boot 3.2.5. The frontend is a React 18 single-page application written in TypeScript and built with Vite. PostgreSQL is the transactional database, Flyway owns schema migration, and Docker Compose provides the reproducible local database and optional backend container. The implementation follows the modular-monolith design established in Chapter 3: domain packages share one backend process while retaining controllers, services, repositories, entities, and DTOs for each functional area.

*Table 4.1. Main implementation technologies*

| Layer | Technology | Implementation purpose |
|---|---|---|
| Backend language/runtime | Java 21 | Domain logic, API, background processing, and integrations |
| Application framework | Spring Boot 3.2.5 | Dependency injection, web MVC, configuration, scheduling, and application lifecycle |
| Security | Spring Security and JJWT 0.12.5 | Stateless bearer authentication, role authorization, and JWT handling |
| Persistence | Spring Data JPA, PostgreSQL 16, Flyway | Entity persistence, queries, constraints, indexes, and versioned migrations |
| Document processing | PDFBox 3.0.2, Apache POI, Tesseract CLI | PDF/DOCX extraction and OCR fallback for images or scanned PDFs |
| API documentation | springdoc-openapi 2.5.0 | OpenAPI documents and Swagger UI |
| Monitoring | Spring Actuator, Micrometer Prometheus registry | Health and HTTP/application metrics |
| Frontend | React 18.3, TypeScript 5.9, React Router 7, React Query 5 | Role-based pages, typed API integration, navigation, and server-state queries |
| Visualization and UI | Recharts and Lucide React | Charts and interface icons |
| Build and test | Maven, npm/Vite, JUnit, Testcontainers, Playwright | Compilation, automated backend tests, integration environment, and browser E2E tests |

The backend package root is com.careerfit.backend. Domain packages include auth, candidate, cv, job, employer, matching, recommendation, application, feedback, automation, notification, analytics, admin, audit, and settings. Shared response models,

exceptions, text utilities, validation, security filters, and configuration are placed in common or configuration packages. This structure makes the dependency direction visible without adding network boundaries between modules.

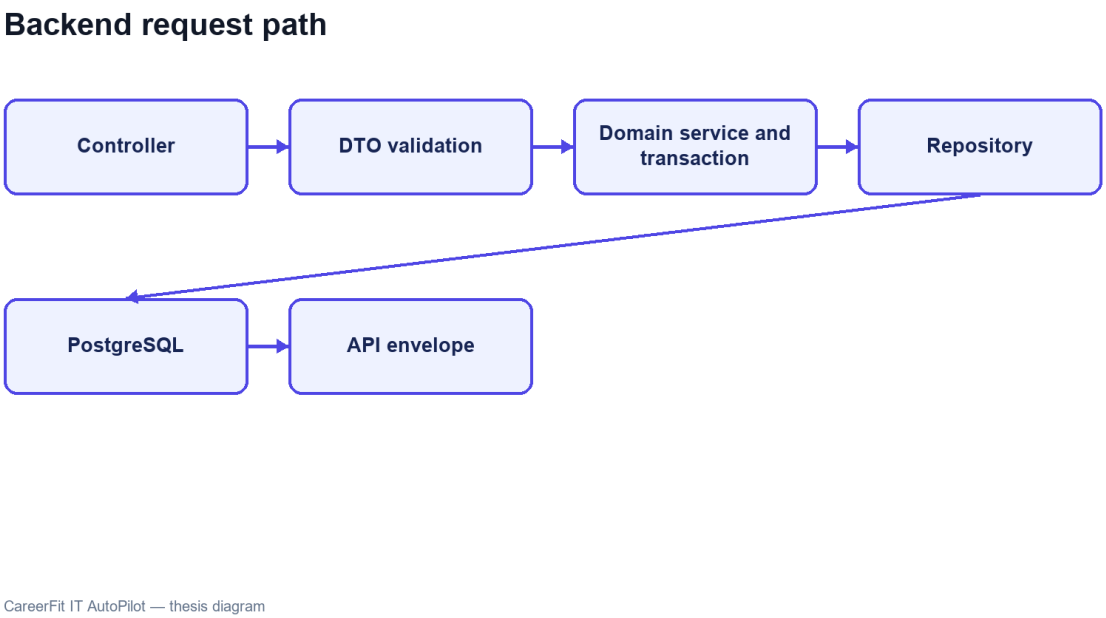



*Figure 4.1. Backend module structure and request flow*

## 4.2 Authentication and Authorization Implementation

### 4.2.1 Login and JWT Processing

AuthController exposes registration, password login, passwordless request/verification, and current-account operations. AuthService validates account state and credentials and delegates token creation to JwtService. Passwords are stored with BCryptPasswordEncoder. A successful login returns an access token and a user DTO containing the ID, email, full name, role, and verification state.

Each authenticated request passes through JwtAuthenticationFilter, a OncePerRequestFilter. The filter requires the Bearer prefix when an Authorization header is present, validates token structure and signature, extracts subject and role, rejects roles outside Candidate, Recruiter, and Administrator, and reloads the account from the database. Reloading prevents a cryptographically valid token from continuing to authorize a deleted or suspended account. The filter then creates a Spring Security authentication object with the corresponding ROLE_* authority.

UserIdResolutionFilter runs after JWT authentication. It resolves the persisted account ID from the authenticated email and stores the UUID as the userId request

attribute. Controllers can therefore pass a stable identifier to services without accepting a user ID supplied by the browser. The implementation currently performs a user lookup in both filters; this is straightforward but creates duplicate database reads that could be consolidated later.

### 4.2.2 URL Rules and Ownership Checks

SecurityConfig uses a stateless session policy. Public access is limited to selected authentication operations, public job/employer/analytics GET routes, similar-job retrieval, health/metrics/OpenAPI routes, and email-action redemption. Candidate routes cover CV, matching, candidate profile, and applications. Recruiter routes cover owned jobs, candidate discovery, application review, employer profile, and export. Administrative routes require the Administrator role. Automation routes require authentication, with policy ownership resolved in services.

Role checks are necessary but insufficient for resources addressed by UUID. ApplicationService, for example, verifies that a requested CV belongs to the authenticated Candidate and that a Recruiter owns the Job before listing applicants, changing status, or inviting a candidate. The same pattern is required across identifier-based services. Errors are returned through shared security and application error writers so that 401 unauthenticated and 403 forbidden responses remain distinguishable.

The configuration disables CSRF because bearer tokens are used instead of a server-side browser session. CORS origins are loaded from application configuration. Frame denial, Content Security Policy, and HSTS headers are configured. HSTS is effective only when the application is served through HTTPS, so the local HTTP runtime does not itself demonstrate transport security.

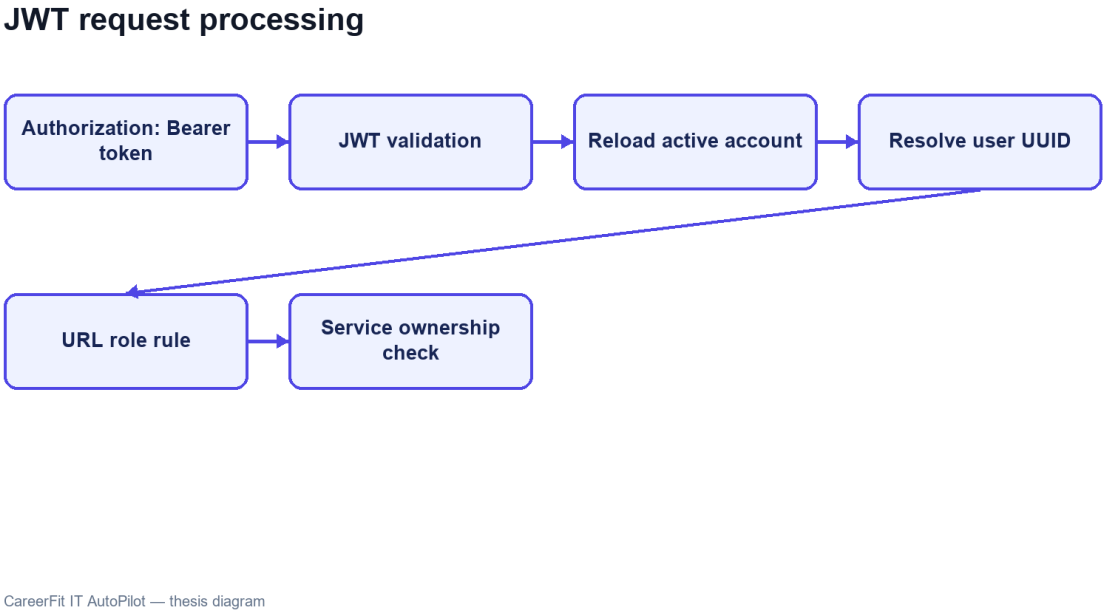



*Figure 4.2. JWT authentication and authorization boundaries*

## 4.3 CV Ingestion and Validation

### 4.3.1 Upload and Manual Creation

CvController provides multipart upload, manual creation, CV listing, details, processing status, default selection, and deletion. CvIngestionService orchestrates the processing pipeline. Uploaded files are validated, stored through StorageService, and represented by a CV entity whose status makes background progress observable. Manual CV data is converted into labeled raw-text sections such as title, seniority, experience, skills, education, projects, certifications, and languages, after which it uses the same normalization and vectorization path.

*Table 4.2. CV processing states*

| State | Meaning | Typical transition |
|---|---|---|
| UPLOADED | Metadata and source have been accepted | Upload/manual creation → validation |
| VALIDATING | File/content and quality checks are executing | Valid input → processing; hard error → failed |
| PROCESSING | Extraction, normalization, and vector creation are executing | Vector saved → scoring done |
| SCORING_DONE | CV vector is available and matching can be queried | Matching runs asynchronously against active jobs |
| FAILED | Processing cannot continue | Failure reason is persisted and audited |

### 4.3.2 File Validation and Extraction

PdfExtractionService accepts PDF, PNG, JPG/JPEG, and DOCX. It checks the extension, MIME type, and leading magic bytes rather than trusting the filename alone. Empty input and mismatched content are rejected. PDFBox first attempts embedded-text extraction. Apache POI extracts DOCX text. Images use OCR directly. For a PDF with fewer than 50 extracted characters, the service renders up to the configured maximum number of pages and invokes a configurable Tesseract command. The default OCR language set is vie+eng, the default maximum is eight pages, and each OCR process has a timeout.

Text shorter than 50 characters after extraction/OCR is a hard failure. Text between 50 and 199 characters is accepted as sparse and logged as a warning. Encrypted PDFs are rejected. Image dimensions are bounded to reduce excessive memory use, and temporary OCR files are deleted after completion. These controls reduce malformed-input risk, although production deployment would additionally require malware scanning and stronger resource isolation.

QualityValidationService produces structured signals for questionable CV or JD content, including seniority/experience contradictions and salary/content inconsistencies. The design separates blocking errors from warnings so that the user can correct poor-quality data without treating every imperfection as a processing failure.

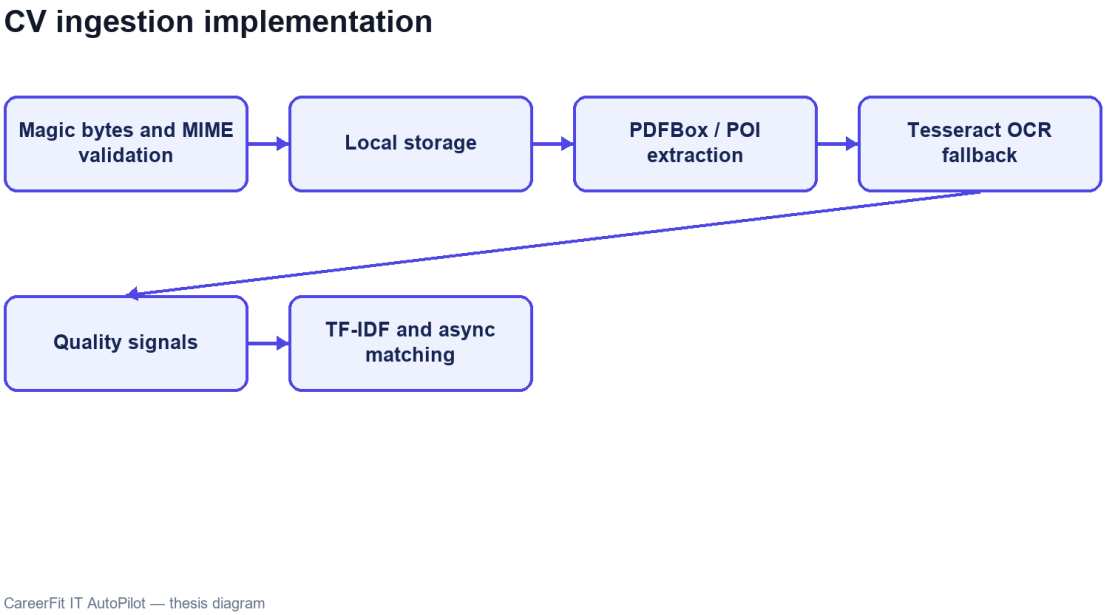



*Figure 4.3. CV ingestion implementation pipeline*

## 4.4 Text Normalization and TF-IDF Implementation

### 4.4.1 Text Normalization

TextNormalizationService removes HTML tags, replaces characters outside its English/Vietnamese alphanumeric pattern, lowercases text, splits on whitespace, removes tokens shorter than two characters, and filters a language-specific stop-word set. Language detection is a lightweight heuristic: text containing more than five Vietnamese diacritic characters is classified as Vietnamese; otherwise it is treated as English.

This implementation is deterministic and easy to inspect, but it is not a linguistic word segmenter. Vietnamese multi-syllable skills can be split into independent tokens, and punctuation-containing technology names such as C++, C#, .NET, or Node.js may lose distinguishing characters. These limitations are important when interpreting matching errors and motivate a future domain-aware tokenizer or controlled alias dictionary.

### 4.4.2 Static Corpus and Vector Construction

TfIdfService builds its IDF map once at application startup from a static seed corpus of representative IT terms. The corpus groups programming languages, frameworks, databases, cloud and DevOps technologies, architecture, testing, security, data/ML, collaboration, seniority, roles, and common Vietnamese IT words. A static corpus prevents scores from shifting whenever a user uploads one CV or adds one Job.

For a token list, term frequency is the token count divided by the total number of tokens. The implemented smoothed IDF is:

$$idf(t) = log(1 + N / (1 + df(t))).$$

An unknown term receives log(1 + N), which treats it as rare and informative. The TF-IDF vector is persisted as JSON and reused during scoring. Cosine similarity iterates over the smaller vector for the dot product, computes both Euclidean magnitudes, and returns zero when either vector is empty.

The unknown-term policy has a practical trade-off. It preserves project-specific technology terms absent from the seed corpus, but a rare misspelling can receive the same high IDF treatment. Input validation, alias normalization, corpus versioning, and

evaluation on realistic data are therefore necessary before treating the score as stable beyond the academic environment.

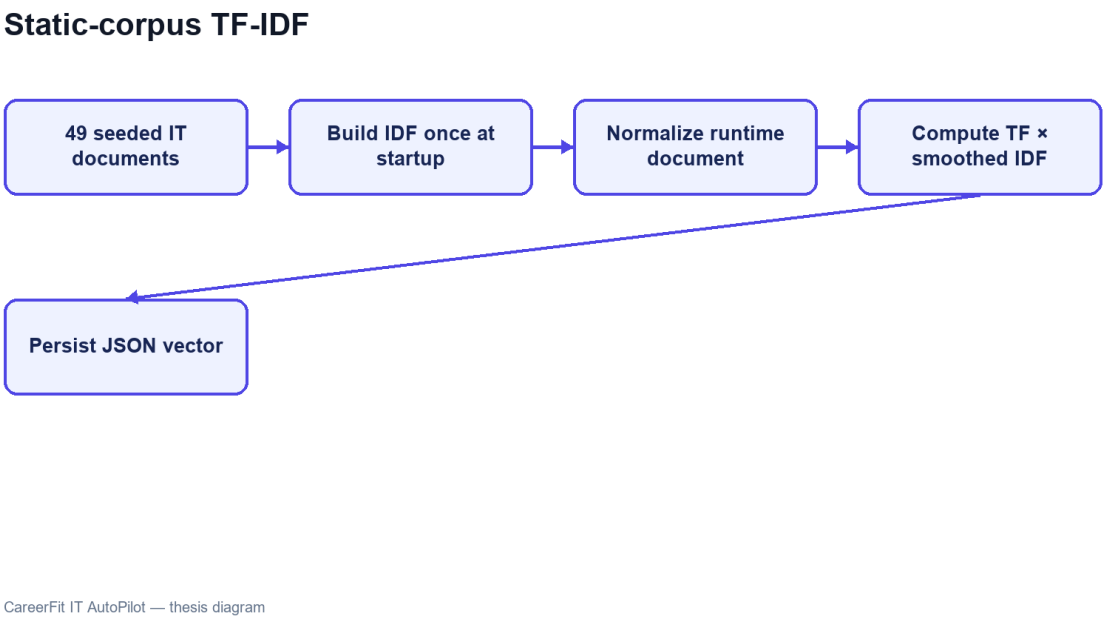



*Figure 4.4. Seed-corpus initialization and TF-IDF construction*

## 4.5 Matching and Recommendation Implementation

### 4.5.1 Scoring Service

ScoringService loads the CV vector from extractedTermsJson. For the Job, it prefers learnedProfileVectorJson when a non-empty learned vector exists; otherwise it uses the original tfidfVectorJson. Cosine similarity becomes the raw score, which is multiplied by 100 and rounded to two decimal places. The raw score is stored with six decimal places.

*Table 4.3. Implemented score interpretation*

| Condition | Stored/displayed result |
|---|---|
| Score below 70 | LOW label |
| Score from 70 to below 90 | MEDIUM label |
| Score at least 90 | HIGH label |
| Score from 35 to below 75 with at least three shared weighted terms | isPotential=true |
| Score from 35 to below 75 with compatible seniority and at least two shared terms | isPotential=true |

Although configuration contains low, medium, and high boundary names, the current assignLabel method uses only the configured medium value (70) and high value (90); the configured low maximum (40) does not create a separate transition. POTENTIAL exists in the enum and schema, but current scoring returns LOW,

MEDIUM, or HIGH and represents potential primarily through the separate boolean and reason. Chapter 5 must test these exact boundaries rather than infer behavior from configuration names.

Match reasons are the five highest-weight Job terms also present in the CV, with Job domain inserted first when available. The potential reason is a fixed explanation about transferable skills and career progression. These reasons are grounded in shared vector terms, but the potential explanation is heuristic rather than a learned causal explanation.

### 4.5.2 Matching Orchestration and Persistence

After CV vectorization commits, AfterCommitExecutor submits MatchingService.scoreAllJobsForCv by CV identifier to the bounded task executor. The service reloads the persisted CV, obtains eligible active Jobs, calls ScoringService for each pair, and creates or updates the unique Matching row. Reasons and potential metadata are serialized as JSON. Database uniqueness on (cv_id, job_id) prevents duplicate pairs, while conflict handling covers concurrent execution. The same boundary is used when a new or updated Job is scored against existing CVs.

When a Recruiter creates or updates a Job, the service builds the Job vector and schedules post-commit scoring. MatchingQueryService shapes Candidate cards and recruiter-oriented results, including pagination and metadata for empty, filtered, low-match, or tied-score states. CandidatePortfolioVisibilityService adds portfolio data only when the Candidate enabled showPortfolioAfterApply and has actually applied or moved beyond an invitation-only state; otherwise the response contains an explicit hidden reason. Stable score ordering remains important because rounded scores can be equal.

### 4.5.3 Recommendation Service

RecommendationService serves personalized Job results and similar-job retrieval. Candidate recommendations use profile/preferences and available matching information, while similar jobs can be retrieved publicly. The API and UI intentionally keep recommendations separate from the persisted application workflow. A recommendation can be viewed, skipped, or used to start an application without becoming an application automatically unless an enabled AutoFit policy separately authorizes that action.

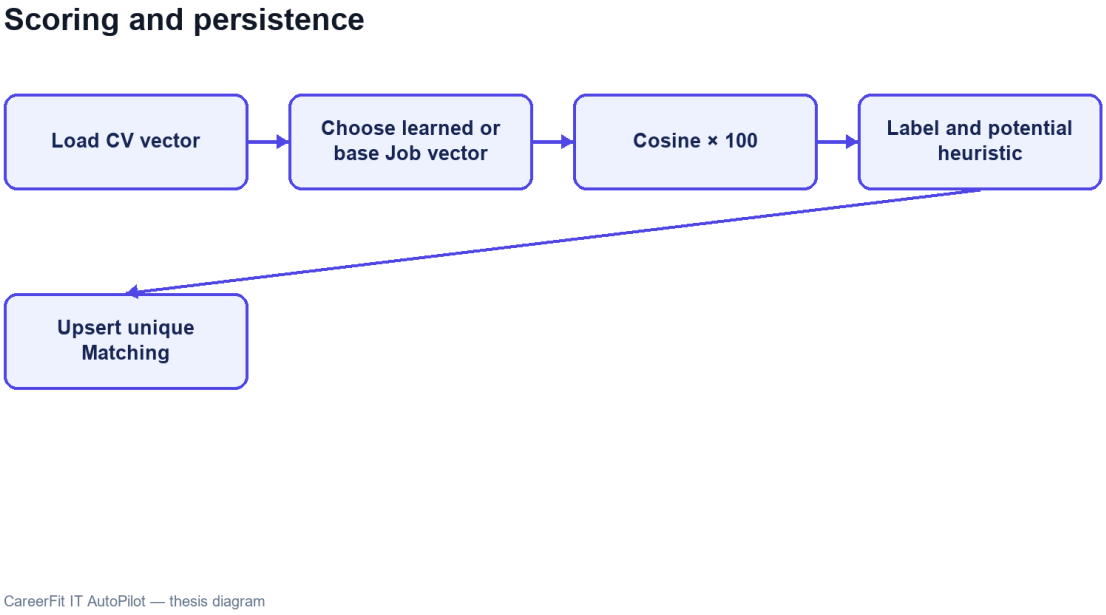



*Figure 4.5. Scoring and Matching persistence flow*

## 4.6 Feedback Learning with Rocchio

FeedbackService.submitFeedback resolves the Matching and authenticated Candidate, verifies that the Candidate owns the associated CV, and then upserts feedback by (matching_id, actor_id). Concurrent uniqueness conflicts are resolved by reloading and updating the existing row. Each event stores actor role, feedback type, and source channel and creates an audit entry. NOT_INTERESTED is stored but does not trigger Rocchio because it can represent preference rather than technical relevance. GOOD_MATCH, POTENTIAL, and BAD_MATCH trigger post-commit Job-vector learning.

RocchioService locks the Job for update and always loads the original Job TF-IDF vector as q. Positive examples are CV vectors attached to GOOD_MATCH or POTENTIAL feedback; negative examples come from BAD_MATCH. The service computes term-wise centroids and applies:

$$q\_new = 1.0q + 0.75\ positive\_centroid - 0.15\ negative\_centroid.$$

Negative weights are removed rather than persisted. The learned vector is stored in learnedProfileVectorJson. Every Matching associated with the Job is then marked needsRecompute=true. AutomationScheduler.recomputeStaleMatchings rescans these rows every 30 minutes and clears the marker only after successful scoring.

Using the immutable base vector and the complete current feedback history makes recomputation idempotent for the same data and parameters. FeedbackService registers learning after transaction commit, so Rocchio reads the persisted judgment rather than racing the initiating transaction. Job locking prevents simultaneous updates from writing independent learned vectors, while integration tests cover the post-commit executor boundary.

*Table 4.4. Feedback handling*

| Feedback | Persisted | Rocchio classification | Immediate learning trigger |
|---|---|---|---|
| GOOD_MATCH | Yes | Positive | Yes |
| POTENTIAL | Yes | Positive | Yes |
| BAD_MATCH | Yes | Negative | Yes |
| NOT_INTERESTED | Yes | Neither | No |

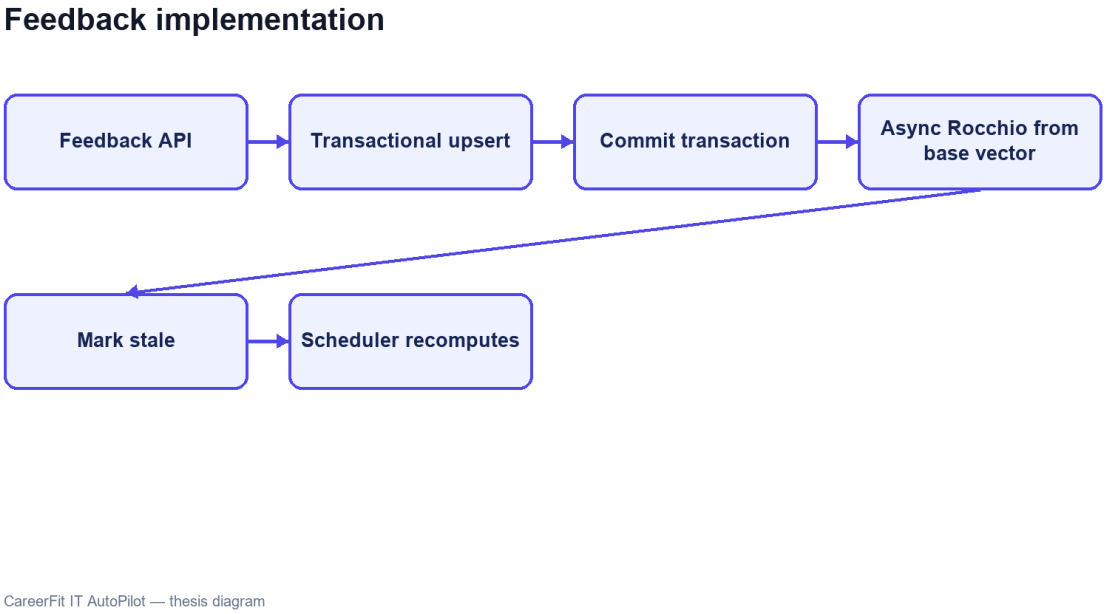



*Figure 4.6. Feedback processing and post-commit learning*

## 4.7 Application and Recruiter Workflow

ApplicationService.submit resolves the authenticated Candidate, requires an active Job, rejects an existing Candidate–Job application, and resolves either the requested CV or the Candidate's default CV. If the corresponding Matching exists, it is attached to preserve the score context at application time. saveAndFlush exposes uniqueness conflicts within the service transaction, which are converted into an HTTP conflict response. The service writes an audit event and requests Candidate and Recruiter notifications.

A Candidate lists owned applications through a paginated response and can withdraw an application unless it is already approved or rejected. Withdrawal changes status to NOT_INTERESTED rather than deleting history. A Recruiter must own the Job to list applicants or update status. Inviting a Candidate creates an INVITED application when no Candidate–Job record exists and returns the existing record when a concurrent or prior action already created it.

The response DTOs combine application state with selected Candidate, CV, and Matching fields without returning persistence entities directly. Portfolio fields are privacy-gated: a Recruiter receives them only after a qualifying application state and when the Candidate has enabled portfolio visibility after applying. List metadata communicates states such as no match, no filtered results, and ready, allowing the frontend to display a meaningful empty state.

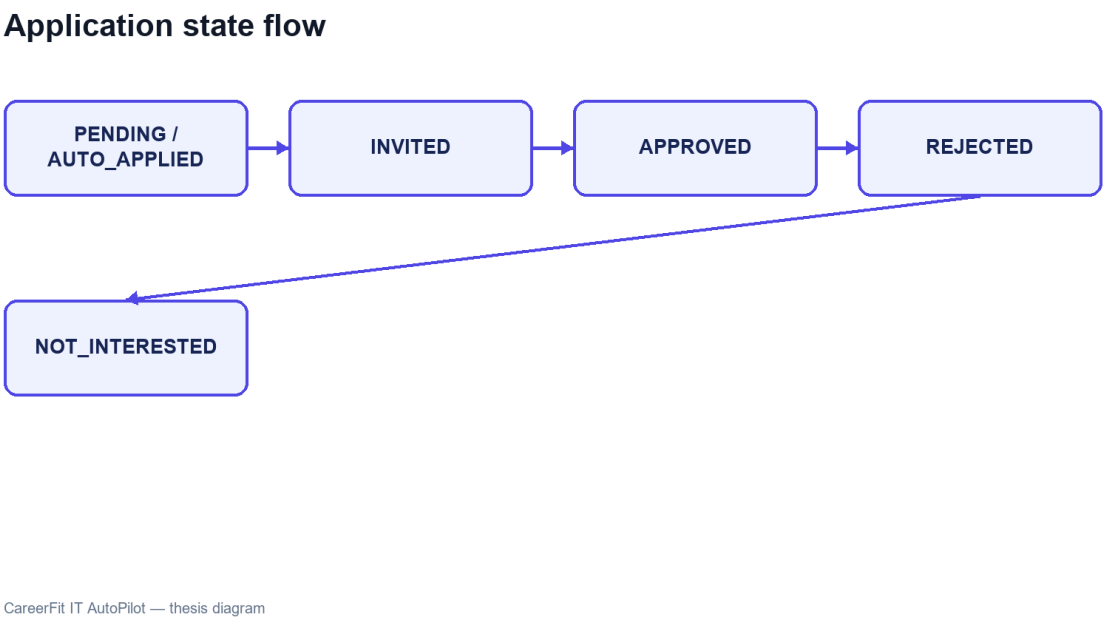



*Figure 4.7. Application and invitation state transitions*

## 4.8 AutoFit and Background Processing

### 4.8.1 Async Executor and Scheduler

AsyncConfig creates a bounded ThreadPoolTaskExecutor using configured core size, maximum size, queue capacity, and thread prefix. AfterCommitExecutor registers CV and Job matching work with Spring transaction synchronization and submits it only after commit. Mail sending and Rocchio learning can also leave the request thread, while persisted CV and Matching states allow clients to observe completion.

AutomationScheduler coordinates five tasks described in Chapter 3. Their delays, cron expressions, and time zone are read from app.scheduler properties. Per-item exception handling prevents one failed candidate or matching record from stopping an entire scan, while logs record the processing outcome.

### 4.8.2 Auto-Apply

AutoApplyService.runForPolicy resolves the policy owner, Candidate, and default CV and requires SCORING_DONE. It reads the top 20 Matches, considers active Jobs at or above the configured threshold, skips existing Candidate–Job applications, and creates at most three automatic applications per run. Database uniqueness is the final concurrency guard. Each created record is marked automatic, audited with the Matching and score, and followed by Candidate and Recruiter notification requests.

The current auto-apply implementation checks the supplied policy's threshold and enablement is handled by the caller selecting enabled policies. It does not independently evaluate every notification control such as quiet hours or email quotas before creating an application. Those settings primarily affect notification policy. This distinction must remain explicit: notification gating and application authorization are related but not identical mechanisms.

### 4.8.3 Notification Guard and Delivery

NotificationPolicyGuard evaluates whether a message is allowed and writes delivery outcomes in independent transactions. It supports deduplication, timing, quota, cooldown, and skipped/failed reasons. After CV scoring, the backend can immediately send a high-match action email, a low-match notice, or a no-match notice; the scheduled scan remains a later safety path and deduplication prevents duplicate delivery. MailService sends asynchronously through SMTP when mail is enabled, while NoOpMailService supports local development without external delivery.

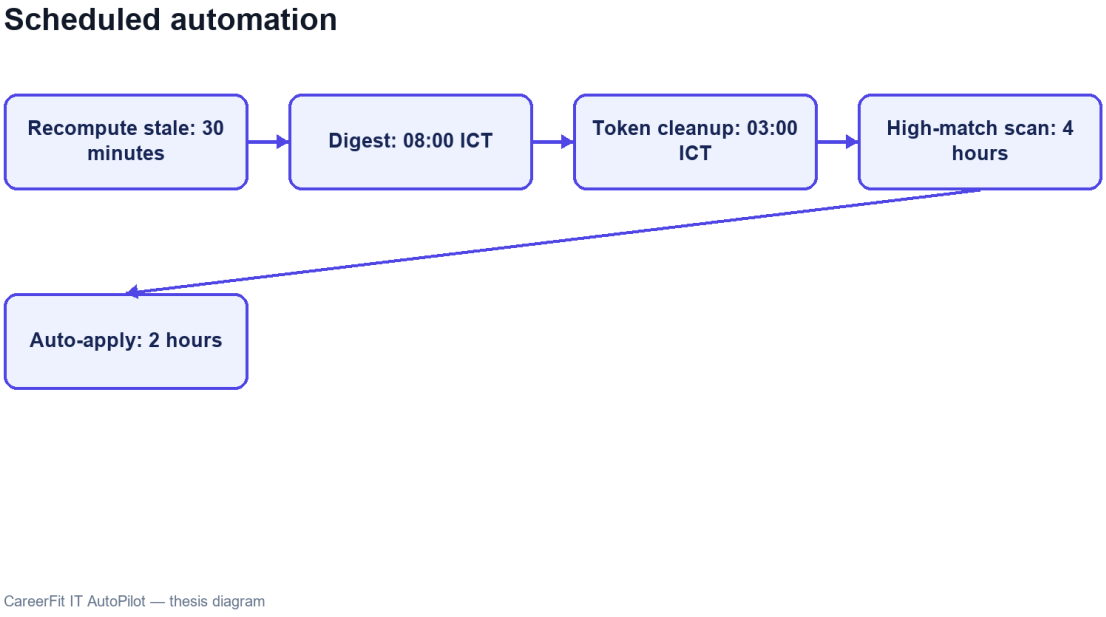



*Figure 4.8. Scheduler and AutoFit execution boundaries*

## 4.9 Email Action and Audit Implementation

EmailActionService generates a 32-character token derived from UUID text for each supported action and stores it with recipient, optional Matching, type, status, and expiry. Match email actions include GOOD_MATCH, POTENTIAL, NOT_INTERESTED, and view links. Digest messages create actions for listed Matches and unsubscribe intent. Tokens remain valid for 72 hours.

EmailActionController exposes a public confirmation and redemption flow because possession of the high-entropy token is the credential. The raw token is sent to the user but only its SHA-256 hash is persisted. GET validates the token and renders a non-mutating confirmation page; POST performs the supported action, records redemption, and rejects unknown, expired, or already-used records.

This implementation removes the earlier state-changing GET and raw-token persistence findings. The remaining production concerns are rate limiting, deployment-specific link origin, secret rotation, mail-delivery monitoring, explicit handling of expired links, and protected audit review.

Audit records are written directly through AuditLogRepository in major services. An audit entry can contain actor type/ID, action, target, result, source channel, and JSON metadata. Examples include CV failure, application submission/withdrawal, candidate invitation, status update, feedback, and auto-apply.

Direct repository calls are simple, but a centralized audit facade could standardize metadata, request IP/user-agent capture, redaction, and failure behavior.

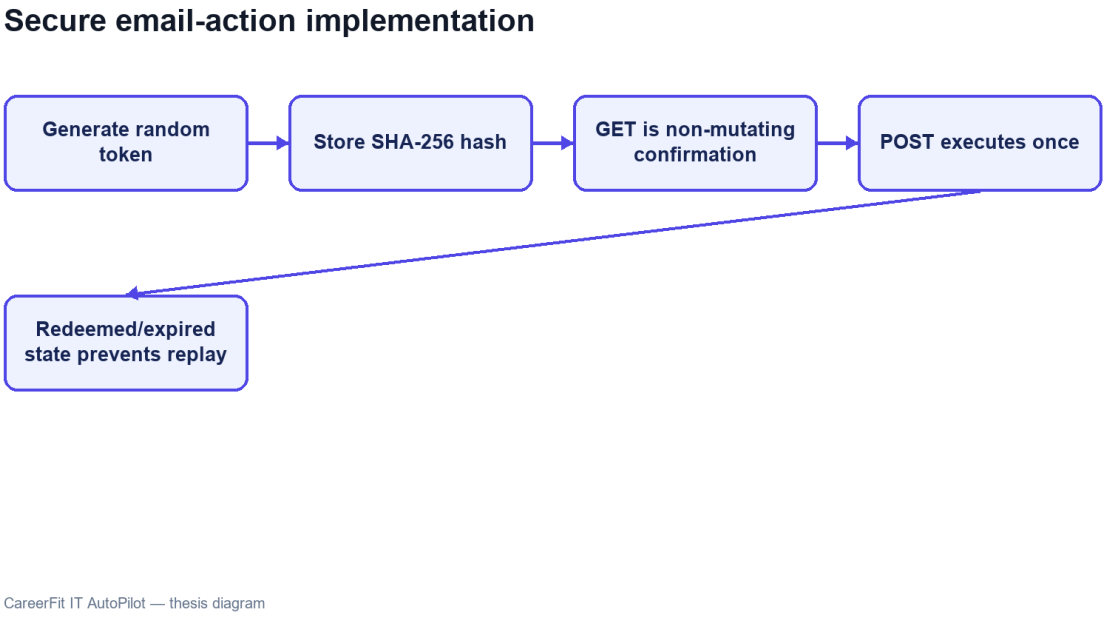



*Figure 4.9. Hashed email-action token and confirm-then-POST flow*

## 4.10 Persistence and API Implementation

JPA entities map the domain tables, while Spring Data repositories provide derived queries, pagination, locking queries, and explicit update/delete operations. Service methods use @Transactional for state changes and readOnly=true for queries where applicable. Flyway migrations V1–V15 create and harden the current schema, including hashed email-action tokens; Hibernate runs with ddl-auto=validate. This prevents application startup from silently inventing schema changes outside migration history.

Database constraints are treated as correctness boundaries rather than only documentation. Unique indexes protect email identity, default CV, Matching, Application, and imported Job hash. Check constraints restrict roles, labels, statuses, source channels, and salary modes. Version columns support optimistic concurrency on mutable core entities. JSONB stores vectors and variable metadata, while relational foreign keys preserve ownership and lifecycle relationships.

Controllers return shared API envelopes and DTOs instead of exposing entities. Validation annotations reject malformed requests near the API boundary, while service exceptions represent not found, forbidden, bad request, and conflict cases. OpenAPI is

generated through springdoc. Public list endpoints and authenticated workspaces use pagination or bounded top-result queries to avoid unbounded payloads.

*Table 4.5. Representative API-to-service mapping*

| API operation | Controller/service responsibility | Persistence effect |
|---|---|---|
| POST /api/cv/upload | Validate source, create CV, start ingestion | CV, file metadata, vectors, Matchings, audit |
| GET /api/matches/me/cards | Resolve Candidate/default CV and map ranked results | Read-only |
| POST /api/matches/{id}/feedback | Upsert feedback and trigger Rocchio | Feedback, audit, learned Job vector, stale flags |
| POST /api/applications | Validate ownership/state and create application | Application and audit |
| PATCH /api/automation/policy | Validate and update owned policy | Automation policy and version |
| POST /api/automation/auto-apply/run-now | Execute the same policy-based service on demand | Automatic applications, audit, notifications |
| GET /api/admin/audit-logs | Filter and paginate administrative audit view | Read-only |

## 4.11 Frontend Integration

App.tsx defines public, Candidate, Recruiter, and Administrator routes. protectedRoute checks the loaded account role, while reload restoration calls /api/auth/me and clears an invalid session. Passwordless login has a dedicated magic-link verification route. Public routes include Jobs and employer details; Candidate routes cover asynchronous CV processing, CV management, recommendations, applications, analytics, automation, and settings; Recruiter and Administrator routes provide their role-specific workspaces. These checks improve navigation, but the backend remains the security boundary.

Frontend/src/lib/api.ts centralizes HTTP access. It reads VITE_API_BASE_URL with /api as the default, attaches the bearer token from sessionStorage, selects JSON headers except for multipart FormData, unwraps the shared response envelope, and throws a normalized error for unsuccessful responses. DTO mapping functions convert backend shapes and labels into UI models. React Query hooks manage loading, refetching, caching, polling, and error state for server data.

The frontend stores the access token and account summary in sessionStorage, limiting persistence to the current browser tab. This still exposes the token to any successful cross-site scripting attack. The Content Security Policy reduces some

exposure, but production hardening should evaluate short-lived tokens, refresh rotation, and HttpOnly secure-cookie alternatives according to the deployed architecture.

API-driven pages no longer substitute mock Job records when requests fail; they expose loading, retryable error, or empty states. The July 18 integration pass connected magic-link completion, session revalidation, CV status polling and management, the dedicated recommendation feed, and the market dashboard to backend contracts. Remaining UI gaps are limited to selected recruiter drill-down, employer self-service, automation pause/resume, telemetry, and administrative maintenance operations.

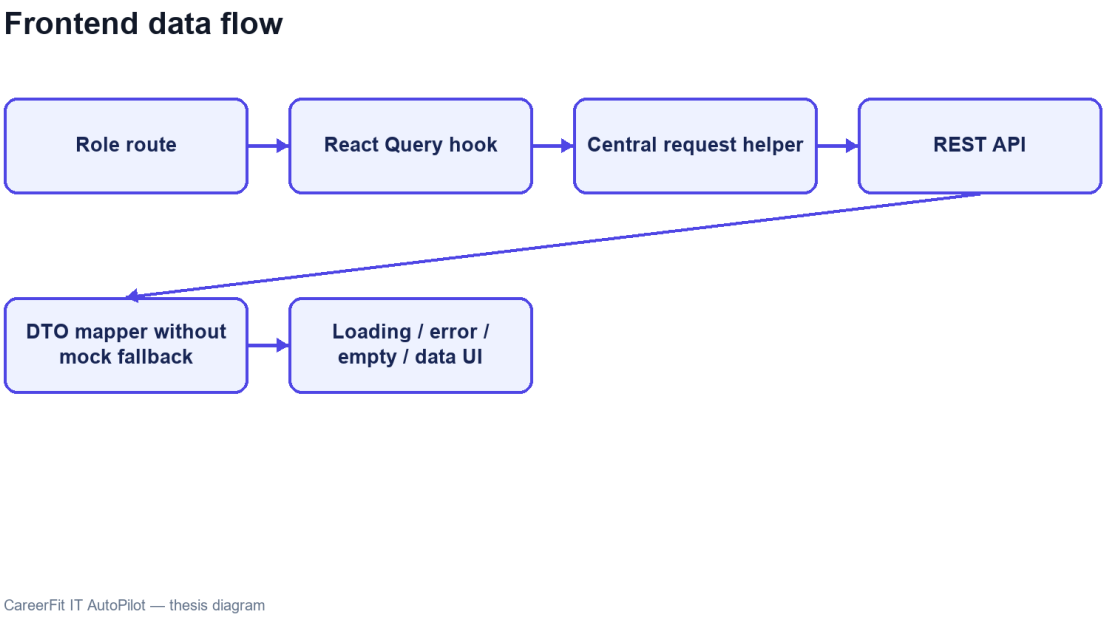
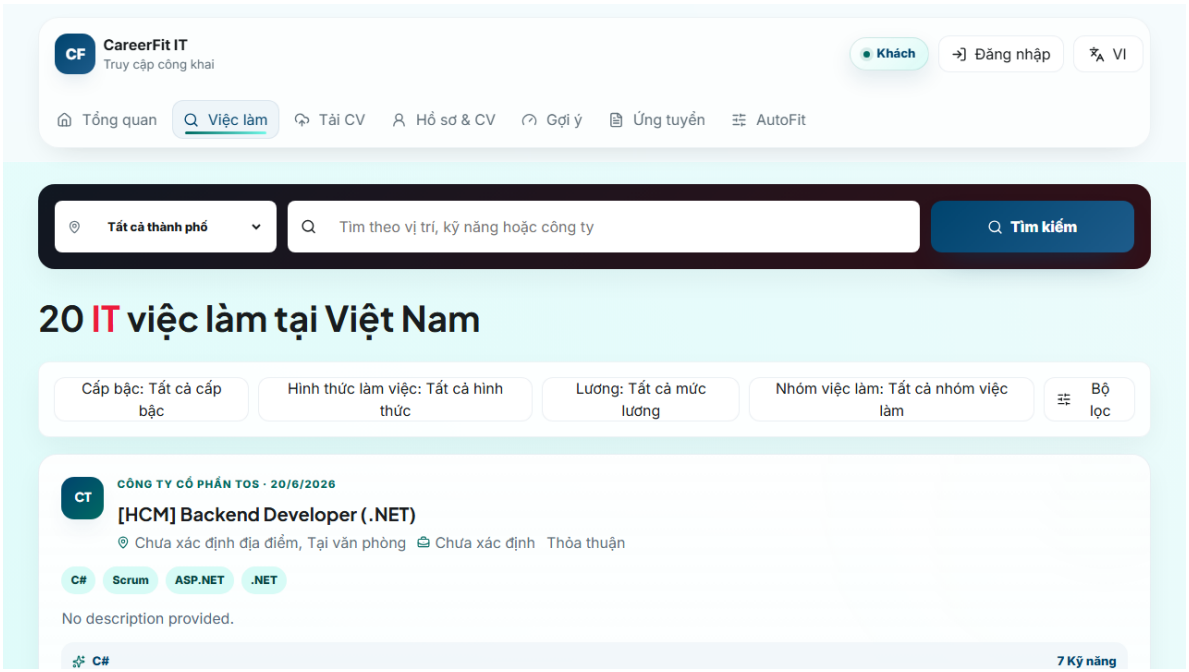
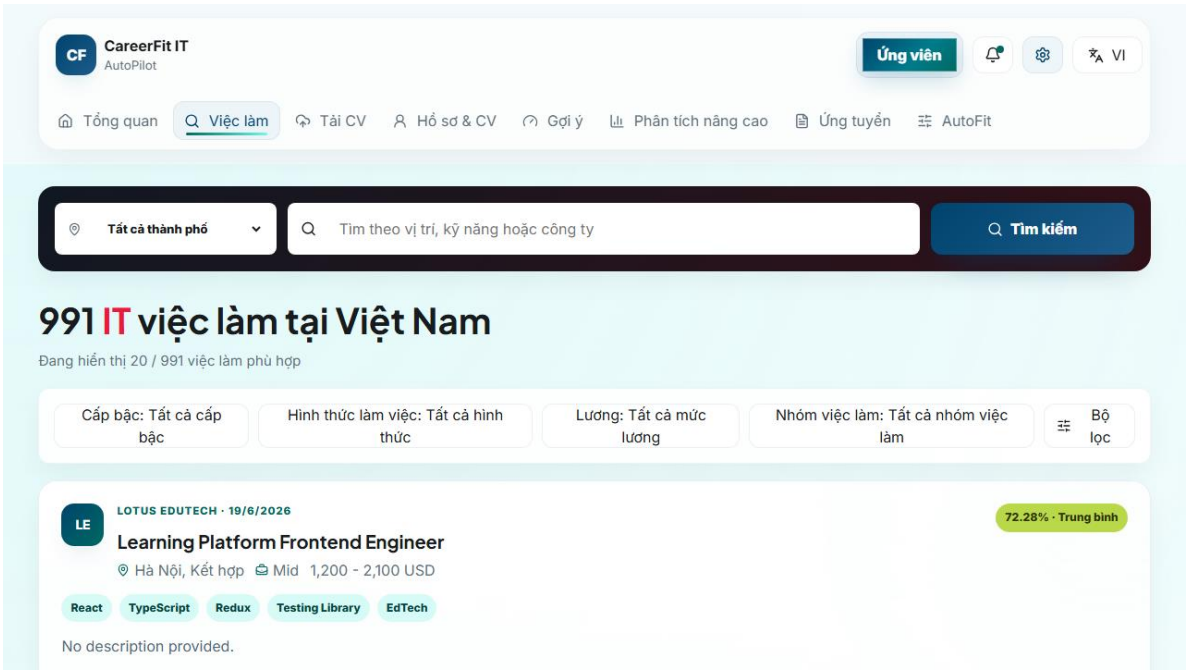


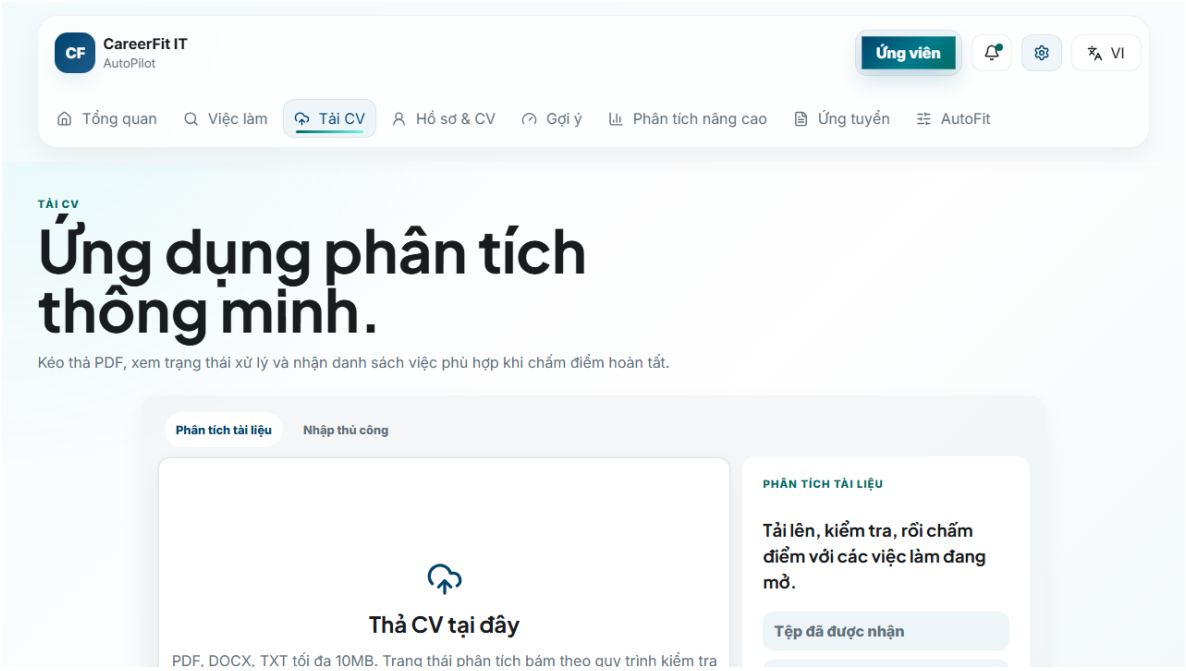

*Figure 4.10. Frontend routes and API data flow*



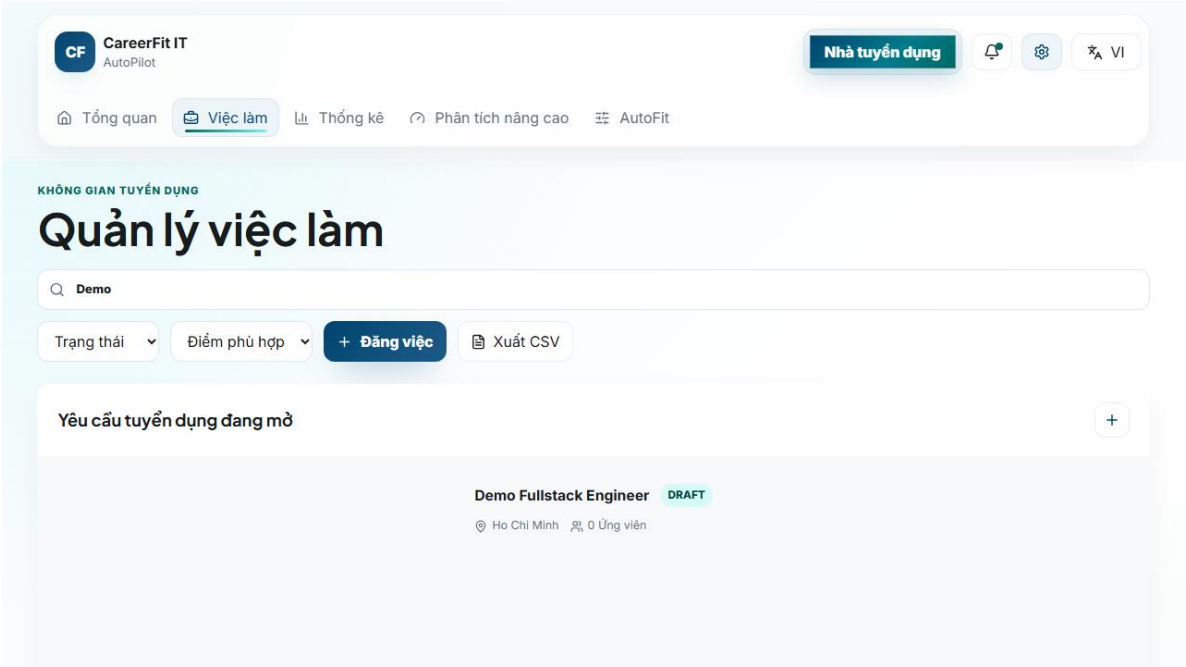

*Screen 4.1. Public Job search*



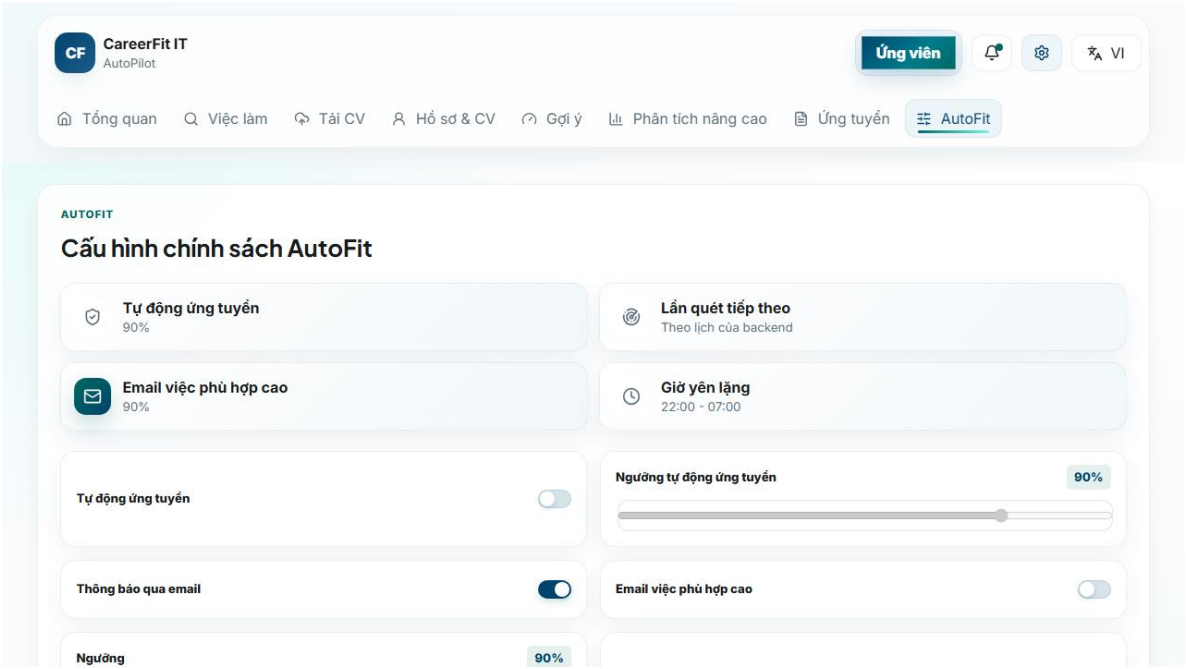

*Screen 4.2. Candidate matching workspace*



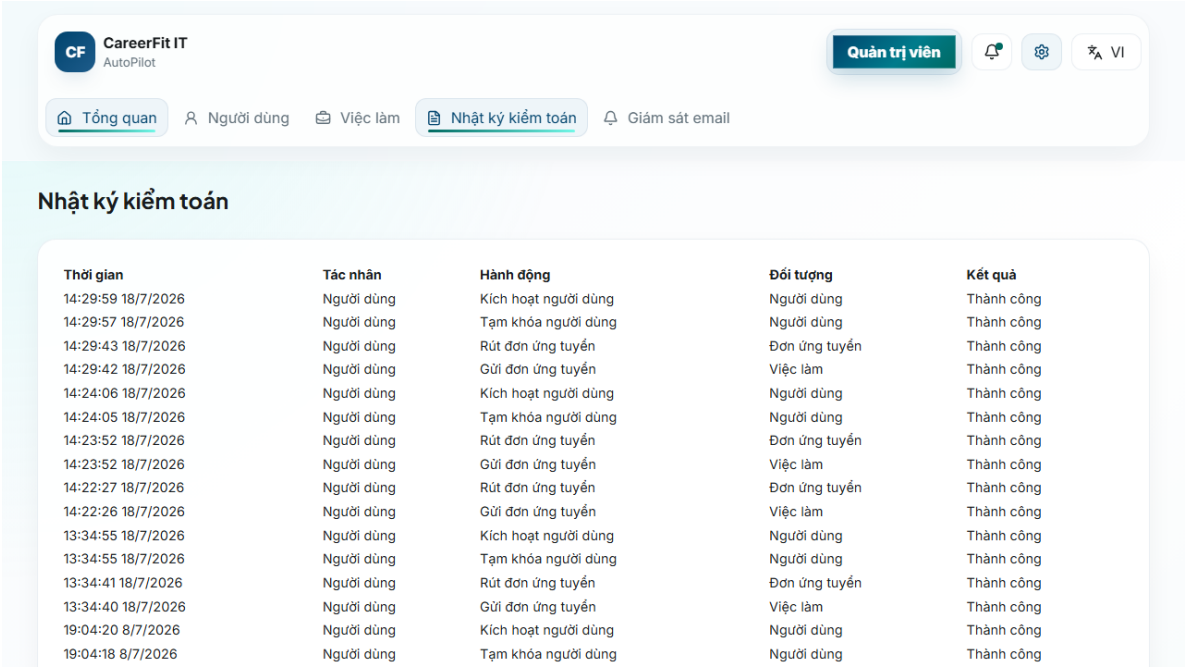

*Screen 4.3. Candidate CV upload interface*




*Screen 4.4. Recruiter candidate workspace*




*Screen 4.5. Candidate AutoFit settings*




*Screen 4.6. Administrator audit view*

## 4.12 Deployment and Observability Implementation

The default local runtime starts PostgreSQL through Docker Compose and can start the backend under the backend profile. Host execution connects to localhost:5433; container execution connects to postgres:5432. The backend image mounts /app/storage/cv, and environment variables override database, JWT, CORS, application URL, mail, OCR, and storage configuration. The frontend development server runs on 127.0.0.1:5173 and uses /api or a configured API base URL.

Actuator exposes health and Prometheus endpoints. Micrometer records HTTP request metrics with the application name tag and enables latency histograms. Console logs include timestamp, thread, level, request ID if present, logger, and message. These facilities provide instrumentation, but the presence of an endpoint is not equivalent to a working monitoring system; Chapter 5 must verify reachability, access restrictions, scrape behavior, and dashboard/alert evidence separately.

Maven builds the backend and can execute JUnit, Spring Security, and Testcontainers tests. The frontend build runs TypeScript checking before Vite bundling. Playwright configurations support local and production-oriented E2E runs. Build and test commands, versions, environment, failures, and generated artifacts must be recorded when Chapter 5 results are finalized.

## 4.13 Implementation Limitations

*Table 4.6. Verified implementation limitations*

| Area | Current limitation | Consequence or required improvement |
|---|---|---|
| Token action | Hashed token storage with GET confirmation and POST execution | Add rate limiting, origin controls, secret rotation, and delivery monitoring |
| Text processing | Whitespace tokenization and punctuation removal | Technology aliases and Vietnamese phrases may be lost |
| TF-IDF corpus | Static hand-curated seed corpus; unseen terms get maximum unknown IDF | Version corpus and test domain drift/misspellings |
| Labels | Configured low maximum is not used as a transition; potential is primarily a boolean | Align configuration names, schema label semantics, UI mapping, and boundary tests |
| Scheduler | All scheduler expressions use app.scheduler properties | Add schedule-boundary and time-zone integration tests |
| Async consistency | Learning is registered after transaction commit | Add production retry, conflict, and failure-recovery policy |

| Area | Current limitation | Consequence or required improvement |
|---|---|---|
| Frontend session | JWT and account summary use sessionStorage | Evaluate HttpOnly cookie or refresh-token architecture and strengthen XSS controls |
| Frontend mapping | API-driven views no longer substitute mock Job fields | Maintain contract tests to prevent fallback data from returning |
| File storage | Local path or Docker volume | Add malware scanning, encryption, backup, retention, and object storage for production |
| Audit | Services write audit rows directly | Centralize redaction, request context, schema conventions, and integrity policy |

These limitations are included because the purpose of an implementation chapter is to explain the system that exists, not an idealized version. They also provide concrete hypotheses and negative cases for Chapter 5.

## 4.14 Chapter Summary

This chapter explained how the design is implemented in Spring Boot, React, PostgreSQL, and Flyway. It covered security, post-commit CV and Job processing, TF-IDF scoring, Candidate feedback with Rocchio, privacy-gated portfolios, application workflows, AutoFit notifications, hardened email actions, frontend integration, and runtime monitoring. Chapter 5 evaluates these parts using refreshed test and runtime evidence.

# CHAPTER 5. EXPERIMENTAL EVALUATION

## 5.1 Evaluation Objectives

The evaluation was designed to answer four questions. First, does the lexical scoring and Rocchio feedback pipeline behave deterministically and change holdout rankings in the intended direction under a controlled causal scenario? Second, do the backend modules and database migrations satisfy their automated unit, integration, security, and contract tests? Third, can the main Guest, Candidate, Recruiter, and Administrator workflows execute through the integrated browser application? Fourth, what can be observed about local API latency, authorization boundaries, health, and monitoring without overstating the evidence as production validation?

The evaluation separates algorithmic effectiveness, software correctness, browser workflow acceptance, security checks, and operational health. A pass in one category does not imply a pass in another. In particular, the controlled Rocchio benchmark is not a measurement of hiring quality on organic production data, and a successful backend test suite does not prove that browser, monitoring, or deployment behavior is complete.

Backend, algorithm, frontend build, and Chromium test results were refreshed on July 18, 2026 (ICT). The observed Git HEAD was 65318fb0e0978574c9a04d9e54aecca5ba1eb241, but the worktree contained additional modifications and untracked files. Therefore, the results describe the evaluated working tree and are not reproducible from the commit hash alone unless the complete working-tree state is preserved.

## 5.2 Evaluation Environment

*Table 5.1. Evaluation environment*

| Component | Observed version or configuration |
|---|---|
| Host operating environment | Windows, PowerShell |
| Java | 21.0.1 |
| Backend framework | Spring Boot 3.2.5 |
| Maven | Repository Maven Wrapper |
| Node.js / npm | 20.16.0 / 10.8.1 |
| Frontend build | Vite 6.4.3, React 18.3, TypeScript 5.9 |
| Docker / Compose | Docker Desktop server 29.5.3 / Compose v5.1.4 |

| Component | Observed version or configuration |
|---|---|
| Test database | PostgreSQL 16.14 through Testcontainers or Compose |
| Testcontainers | 1.21.4 |
| Browser E2E | Playwright 1.61, Chromium project |
| Local backend | Host process on port 8080 connected to Compose PostgreSQL at localhost:5433 |
| Local frontend | Vite development server on 127.0.0.1:5173 |

Docker Desktop was initially unavailable and was started before database-dependent verification. Testcontainers created isolated PostgreSQL containers for backend integration tests. Browser E2E used the existing local CareerFit database, which contained imported and seeded data. The public Job API reported 991 Jobs during the runtime check. This database is not the same dataset as the controlled algorithm benchmark.

The original command catalog is recorded in evidence/CHAPTER5_EVIDENCE_20260703.md. For the July 18 refresh, current Surefire XML reports, Playwright output, the production-build output, backend runtime logs, and evaluation/result.json provide the underlying artifacts.

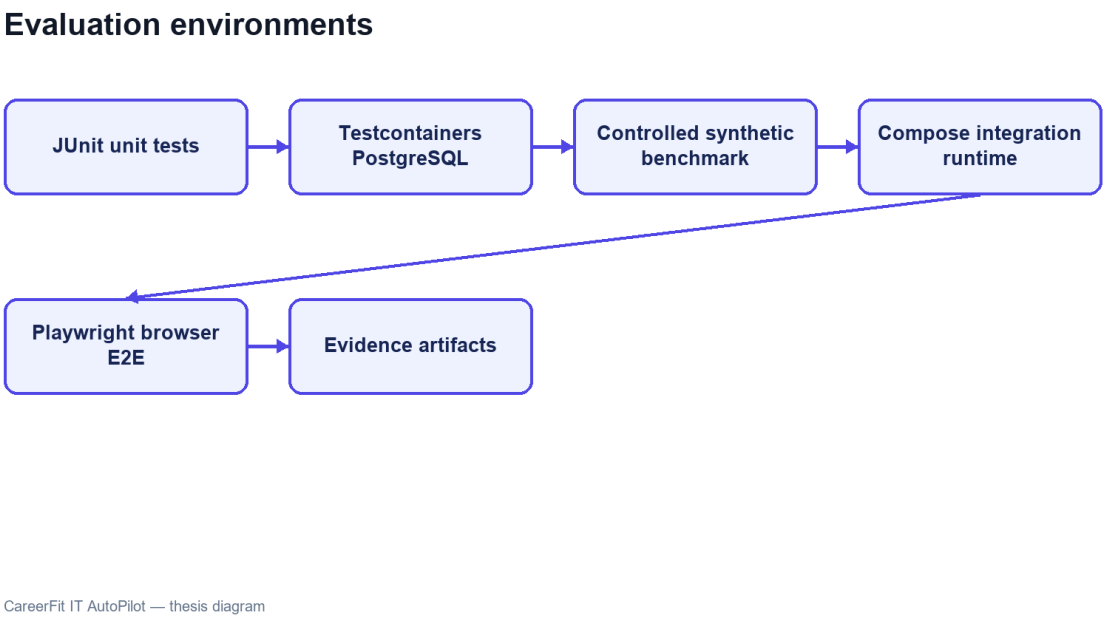



*Figure 5.1. Evaluation environments and evidence sources*

## 5.3 Dataset and Evaluation Design

### 5.3.1 Controlled Algorithm Dataset

AlgorithmEvaluatorTest loads evaluation/controlled-dataset.json. The dataset contains 50 Jobs and 100 unique CVs across IT domains. It defines 300 training pairs and 300 holdout pairs. The file hash observed during every repeated run was 6e935639ba6d3290dca8ad91a35d714c5e30c7e69a59af23ddbcf89fcc5cc2f2.

The benchmark is intentionally synthetic. Each Job is associated with a training-positive CV and a separate holdout-positive CV sharing a latent skill that is not emphasized in the original JD. The baseline ranks candidates using the original lexical representation. A positive feedback event supplies evidence from the training CV; Rocchio updates the Job vector; and the system reranks candidates. Metrics are then computed on holdout relevance rather than using the same CV that generated feedback as the only success case.

This design tests causal behavior: whether a defined relevance signal can move a related holdout item upward. It does not reproduce organic recruiter behavior, naturally imbalanced applicant pools, demographic variation, adversarial CV writing, or changing labor-market terminology. The deliberately planted latent feature also makes a large improvement possible and should not be interpreted as the expected effect size in production.

### 5.3.2 Local Runtime Data

The integrated runtime used Flyway seed data and previously imported Job records. It supports realistic API volume and complete application workflows, but it is not a labeled relevance dataset. Consequently, it is suitable for functional/E2E and small latency checks, not for calculating ranking precision or fairness. E2E uses demo Candidate, Recruiter, and Administrator accounts and changes local database state.

## 5.4 Backend Automated Testing

The complete backend suite was executed through the Maven Wrapper using .\mvnw.cmd test. It compiled 127 application source files and 22 test source files, started Testcontainers, validated and applied 15 Flyway migrations, and executed 20 Surefire test suites.

*Table 5.2. Backend automated test results*

| Measure | Result |
|---|---|
| Test suites | 20 |
| Tests | 72 |
| Failures | 0 |
| Errors | 0 |
| Skipped | 0 |
| Aggregated Surefire test time | 244.659 s |
| End-to-end command wall time | Approximately 266 s |

The suites covered application context startup, API contracts, post-commit execution, authentication, auto-apply, Candidate profile and CV ingestion, feedback authorization, Job service, matching batches, document extraction, quality validation, Rocchio, scoring, settings, TF-IDF, production configuration, and security hardening. AlgorithmEvaluatorTest was included in the same complete suite.

All 72 registered JUnit tests passed in the July 18 run, with no failures, errors, or skips. Flyway 9.22.3 still warned that PostgreSQL 16.14 is newer than the highest version it reports as tested, and some negative tests intentionally logged handled exceptions. The final benchmark log contained no StaleObjectStateException or build failure.

## 5.5 Controlled Algorithm Results

The benchmark used Rocchio parameters $\alpha$=1.0, $\beta$=0.75, and $\gamma$=0.15. Coverage remained 1.0 before and after feedback. Table 5.3 reports the fresh artifact values.

*Table 5.3. Baseline and Rocchio results on the synthetic holdout scenario*

| Metric | Baseline | After Rocchio | Delta |
|---|---|---|---|
| Precision@5 | 0.012000 | 0.172000 | +0.160000 |
| Recall@5 | 0.060000 | 0.860000 | +0.800000 |
| nDCG@3 | 0.030000 | 0.830000 | +0.800000 |
| nDCG@5 | 0.037737 | 0.837737 | +0.800000 |
| nDCG@10 | 0.050424 | 0.850424 | +0.800000 |
| MRR | 0.058755 | 0.842665 | +0.783910 |
| HitRate@5 | 0.060000 | 0.860000 | +0.800000 |
| Coverage | 1.000000 | 1.000000 | 0 |

The increase across position-sensitive metrics shows that the controlled positive feedback moved relevant holdout CVs toward the top of the ranking. Precision@5

reaches 0.172 after feedback, so the top-five lists are not composed entirely of labeled relevant items. Recall@5 and HitRate@5 reach 0.86 in the designed scenario, while 14 percent of query cases still do not place the expected holdout item within the first five results.

The July 18 full-suite run used dataset hash 6e935639ba6d3290dca8ad91a35d714c5e30c7e69a59af23ddbcf89fcc5cc2f2. It produced baseline nDCG@5 of 0.037737056145, Rocchio nDCG@5 of 0.837737056145, and a delta of 0.80. AlgorithmEvaluatorTest completed in 212.2 seconds inside the full suite. These values describe the current code and dataset; they replace the earlier July 3 metric snapshot.

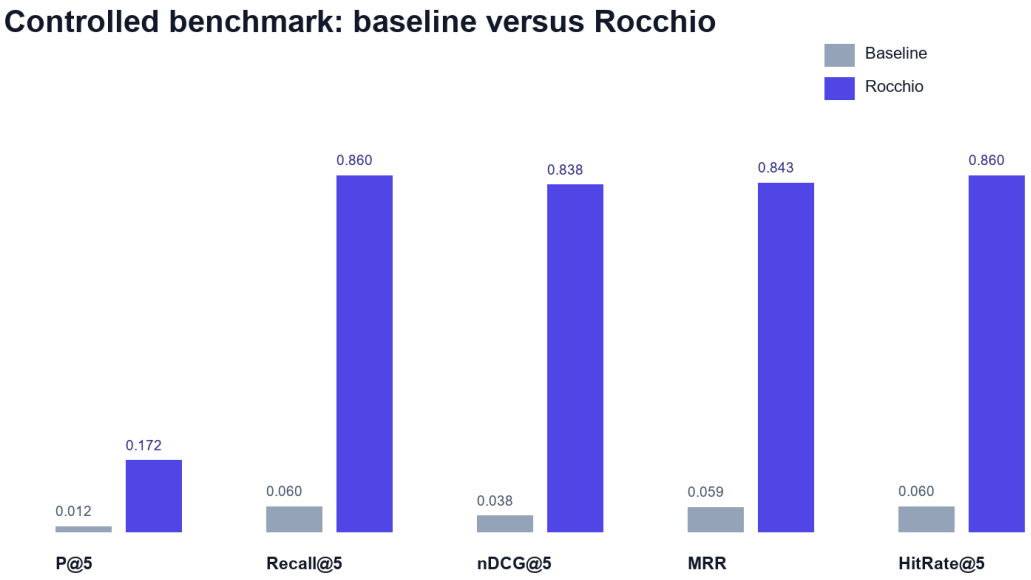



*Figure 5.2. Baseline and Rocchio benchmark metrics at K = 5*

## 5.6 Concurrency Observation in Benchmark Runs

The final isolated benchmark completed without StaleObjectStateException. Feedback learning is registered after transaction commit, and benchmark setup removes prior Matchings in bulk and clears the persistence context before recomputation. This makes the repeated baseline-versus-learned measurement deterministic within the controlled test environment.

Earlier benchmark repetitions exposed a background persistence conflict even when foreground assertions passed. The current run no longer logged that exception because feedback learning and matching begin after the initiating transaction commits

and the benchmark clears prior Matching state before recomputation. This distinction remains important: test assertions and background operational cleanliness must both be checked.

The implemented correction controls asynchronous transaction timing and checks the final logs for background failures. Production work should still define retry, conflict, timeout, and recovery policies, and CI should continue to fail when uncaught background exceptions appear even if foreground JUnit assertions pass.

## 5.7 Frontend Build Evaluation

npm run build performs TypeScript checking with no output emission and then builds the production assets with Vite 6.4.3. The July 18 command completed successfully and transformed 2,417 modules.

*Table 5.4. Frontend build artifacts*

| Artifact | Uncompressed size | Gzip size |
|---|---|---|
| index.html | 1.37 kB | 0.67 kB |
| Main CSS | 65.09 kB | 12.89 kB |
| Icons chunk | 19.25 kB | 4.48 kB |
| Query chunk | 39.60 kB | 12.18 kB |
| React chunk | 180.98 kB | 59.54 kB |
| Application chunk | 217.39 kB | 58.05 kB |
| Charts chunk | 378.44 kB | 112.04 kB |

Manual Rollup chunking separated React, query, chart, and icon dependencies. The largest generated JavaScript chunk was 378.44 kB, below Vite's 500 kB warning threshold. No browser performance profile was collected, so bundle size is reported as build evidence rather than a direct page-load measurement.

## 5.8 Browser End-to-End Evaluation

The Playwright Chromium project ran against the Vite frontend, host backend, and Compose PostgreSQL. All 20 tests across the P0 workflow, backend-contract, and resilience suites passed in 34.1 seconds.

*Table 5.5. Chromium P0 workflow results*

| Scenario | Main verification | Result |
|---|---|---|
| Guest search and Job detail | Public search API, Job cards, detail navigation, login-to-apply control | Passed |

| Scenario | Main verification | Result |
|---|---|---|
| Candidate apply and withdraw | Login, personalized Jobs, application creation, history, withdrawal | Passed |
| Recruiter creates a JD | Recruiter login, creation form, API response, new Job visible | Passed |
| Administrator suspends/reactivates a user | Admin login, user operation, state transition and restoration | Passed |
| Candidate and authentication contracts | Passwordless request, settings persistence, recommendations, CV polling, magic-link completion, and session restore | Passed |
| Resilience and role navigation | Retryable API errors, feedback contract, employer/similar-job data, desktop header, and role-route rendering | Passed |

The result covers the four original role workflows together with passwordless request, settings persistence, recommendations, role-route rendering, CV polling, magic-link completion, session restoration, retryable errors, feedback contracts, employer details, similar Jobs, and desktop navigation. It is not a full UAT study: no independent participants performed tasks, seeded credentials were used, and Firefox and WebKit were not run. The Recruiter flow now removes the Job it creates, although the local database still contains earlier test and imported records.

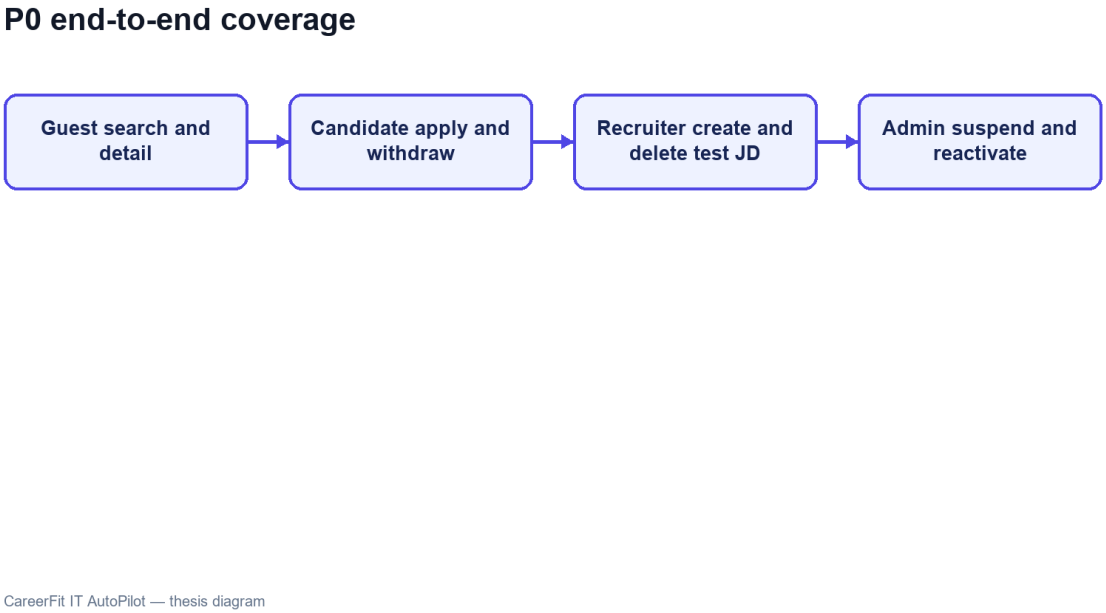



*Figure 5.3. P0 end-to-end workflow coverage*

## 5.9 Security-Oriented API Checks

Small negative checks were executed against the integrated backend to verify public access, missing authentication, invalid authorization scheme, valid Candidate authentication, and role denial.

*Table 5.6. API authorization observations*

| Request | Expected | Observed | Result |
|---|---|---|---|
| Public Job search without token | 200 | 200 | Passed |
| Candidate CV endpoint without token | 401 | 401 | Passed |
| Current-account endpoint with invalid Basic scheme | 401 | 401 | Passed |
| Current-account endpoint with valid Candidate bearer token | 200 | 200 | Passed |
| Administrator dashboard with Candidate bearer token | 403 | 403 | Passed |

These checks confirm selected URL-layer outcomes, not a complete penetration test. They do not evaluate token theft, cross-site scripting, CV malware, rate limiting, secret rotation, SQL-injection tooling, or every ownership boundary. The email-action flow now uses hashed storage and confirm-then-POST redemption, but the broader production-security topics remain outside the verified evidence.

## 5.10 Runtime Health, Monitoring, and Local Latency

The public Job API returned HTTP 200. The aggregate /actuator/health endpoint and the liveness and readiness groups returned HTTP 200 with status UP. Mail health is disabled when application mail is disabled, preventing an intentionally absent local mail provider from making aggregate health misleading. Prometheus metrics remained available in the local evaluation profile.

*Table 5.7. Runtime observations*

| Endpoint or check | Observation |
|---|---|
| /api/jobs/search?page=0&size=20 | 200 |
| /actuator/health | 200, UP |
| /actuator/health/liveness | 200, UP |
| /actuator/health/readiness | 200, UP |
| /actuator/prometheus | 200 |

The final runtime check returned HTTP 200 and status UP for aggregate health, liveness, and readiness. Mail health is disabled when application mail is disabled, so the absence of a local mail provider no longer makes aggregate health misleading. Production monitoring would still require protected component details, alerting, and an external mail check when email is enabled.

Thirty sequential warm requests to the public Job search endpoint produced a mean of 61.79 ms, p50 of 55.20 ms, p95 of 85.11 ms, minimum of 44.99 ms, and maximum of 99.32 ms. This was a single-client localhost sample without concurrent load, controlled warm-up, network latency, repeated batches, or resource monitoring. It demonstrates only that the observed local endpoint responded consistently in this small sample; it is not a throughput or capacity benchmark.

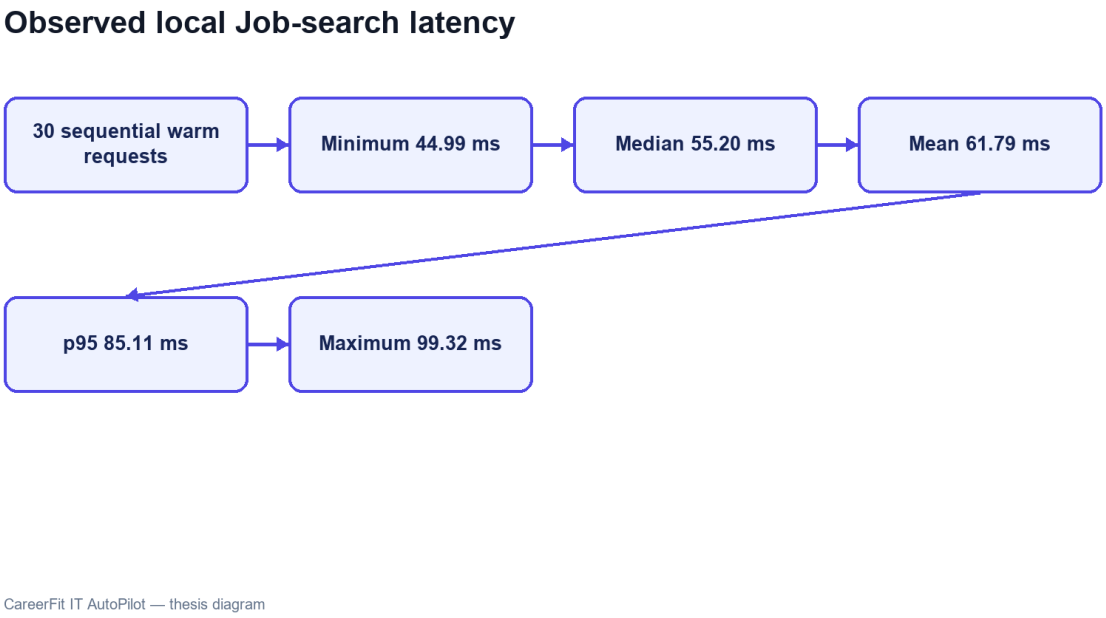



*Figure 5.4. Local Job-search latency distribution*

## 5.11 Evaluation Summary by Research Question

*Table 5.8. Answers supported by current evidence*

| Question | Evidence-supported answer |
|---|---|
| Does Rocchio change holdout ranking in the intended direction? | Yes for the controlled synthetic causal dataset; the same large metric delta was reproduced across three runs. |
| Is the backend automated suite passing? | Yes: 72/72 registered tests passed, and the final benchmark log contained no optimistic-lock exception. |
| Do the main browser workflows execute? | All 20 Chromium workflow, contract, and resilience tests passed; Firefox/WebKit and participant UAT remain incomplete. |
| Are selected authorization boundaries enforced? | The five observed public/authentication/role checks returned expected statuses; this is not comprehensive security validation. |
| Is the runtime fully healthy? | Yes in the evaluated local profile: aggregate health, liveness, readiness, and the core API returned HTTP 200. This is not production monitoring validation. |
| Is performance production-ready? | Not evaluated. Only a small sequential localhost latency sample was collected. |

## 5.12 Threats to Validity

### 5.12.1 Construct Validity

The synthetic benchmark measures retrieval behavior for planted relevance relationships, not hiring success, candidate quality, fairness, or recruiter satisfaction. Matching metrics cannot establish whether a person should be employed. Scripted E2E success measures workflow execution, not usability or user trust.

### 5.12.2 Internal Validity

The benchmark runs inside a Spring context, so asynchronous work and persisted Matching records can affect repeatability if they are not controlled. The final test registers learning after commit, clears previous Matching state before recomputation, and checks logs for background failures. Browser tests use explicit API and UI conditions, avoid relying only on the first seeded Job, and delete the Job created by the Recruiter flow.

### 5.12.3 External Validity

The controlled data lacks organic recruiter judgments, evolving terminology, demographic diversity, adversarial behavior, and production class imbalance. The runtime is one Windows workstation with Docker Desktop, one local PostgreSQL instance, and one Chromium browser. Results cannot be generalized to enterprise load, different browsers, cloud deployment, or a population of real users.

### 5.12.4 Reproducibility

The benchmark dataset hash and generated JSON support algorithm reproduction. Maven Wrapper, Flyway, Testcontainers, package lock, and recorded commands support environment reconstruction. Recruiter E2E cleanup is now automatic, but reproducibility is still weakened by the dirty working tree, mutable local database, imported records, and external web assets. The final thesis release should be associated with a clean commit or archive and immutable evidence bundle.

## 5.13 Chapter Summary

The July 18 evaluation passed all 72 backend tests, completed the frontend production build, and passed all 20 Chromium workflow, contract, and resilience tests. The controlled Rocchio benchmark improved the holdout ranking on the synthetic dataset, and its final log contained no optimistic-lock exception. Aggregate health returned HTTP 200 with status UP. These results support a functioning academic prototype, but not production readiness or proven effectiveness on real recruitment data.

# CHAPTER 6. RESULTS, DISCUSSION AND CONCLUSION

## 6.1 Summary of Results

CareerFit IT AutoPilot was implemented as an IT-focused recruitment prototype with public job discovery, role-based workspaces, CV processing, matching, recommendation, Rocchio feedback, applications, AutoFit policies, email actions, analytics, and administrative monitoring. The backend follows a modular-monolith structure and the user interface is implemented in React.

The refreshed evaluation provides evidence for several technical outcomes. The complete backend suite executed 72 registered tests across 20 suites with no failures, errors, or skips. The frontend passed TypeScript checking and Vite production compilation. All 20 Chromium tests passed across role workflows, backend contracts, and resilience checks, including public search, Candidate application, Recruiter JD creation, Administrator operations, passwordless flow, CV polling, session restoration, and error states.

The controlled Rocchio benchmark demonstrated the intended causal behavior on its synthetic dataset. With 50 Jobs, 100 unique CVs, 300 training pairs, and 300 holdout pairs, nDCG@5 increased from 0.037737 to 0.837737, Recall@5 and HitRate@5 increased from 0.06 to 0.86, and MRR increased from 0.058755 to 0.842665. These results demonstrate adaptation in the constructed latent-skill scenario; they do not estimate effectiveness on organic recruitment data.

The July 18 refresh retained the clean benchmark and aggregate health results, kept the largest frontend chunk below 500 kB, removed API-data fallbacks, expanded Chromium coverage to 20 tests, and refreshed the six interface screenshots. It also verified post-commit matching, Candidate-only feedback authorization, immediate match-result notifications, and portfolio privacy gating. These results support a defensible local demonstration; they do not establish production readiness.

*Table 6.1. Consolidated result status*

| Evaluation area | Supported result | Qualification |
|---|---|---|

| Evaluation area | Supported result | Qualification |
|---|---|---|
| Backend automated tests | 72/72 registered tests passed | Final logs were also checked for background exceptions |
| Controlled feedback benchmark | Large, deterministic improvement after Rocchio | Synthetic causal design; not production effectiveness |
| Frontend build | TypeScript and Vite build passed | Largest JavaScript chunk was 378.44 kB; no Vite size warning |
| Browser workflows | 20/20 Chromium tests passed | Workflow, contract, and resilience coverage; no Firefox/WebKit or independent participant UAT |
| Authorization spot checks | Observed 200/401/403 outcomes matched expectations | Not a complete security assessment |
| Runtime APIs | Core Job API and aggregate/liveness/readiness health returned HTTP 200/UP | Local profile only; no external production monitoring |
| Local latency sample | Mean 61.79 ms and p95 85.11 ms for 30 sequential Job queries | No concurrency, controlled load, or production network |

## 6.2 Achievement of Thesis Objectives

### 6.2.1 Role-Based Recruitment Platform

The objective of providing role-specific workflows was achieved at prototype level. Guests can browse public Jobs and employer information. Candidates can maintain profiles, CVs, and portfolios; inspect matching and recommendation results; apply or withdraw; submit feedback; and configure automation. Recruiters can manage Jobs, inspect applicants and discovered candidates, invite candidates, and update application status. Administrators can monitor users, Jobs, audit records, and email/token state. The Chromium P0 suite confirms representative workflows for all four interface roles, although it does not cover every route or browser.

### 6.2.2 CV and Job-Description Processing

The project implements uploaded and manually created CVs, explicit processing statuses, file-type and magic-byte validation, PDF and DOCX extraction, image and scanned-PDF OCR fallback, sparse-text warning, failure reason persistence, and structured quality signals. Job creation includes structured fields and conditional validation. This objective is supported by source inspection and automated tests for extraction, validation, Job service behavior, and integration contracts.

The implementation remains bounded by local storage, external Tesseract availability, page and timeout limits, and a deterministic but simple tokenizer. It should be described as a functioning ingestion pipeline rather than a universal document-understanding system.

### 6.2.3 Matching and Recommendation

CareerFit implements a static-corpus TF-IDF vectorizer, cosine similarity, normalized score, LOW/MEDIUM/HIGH labels, potential heuristics, and shared-term reasons. CV–JD matching and profile-oriented recommendation remain separate workflows, and Matching state remains separate from Application state. Scoring, TF-IDF, batch isolation, and API contract tests passed.

The objective was achieved as an interpretable lexical baseline. It was not achieved as a semantic equivalence model: punctuation-sensitive technology names, synonyms, Vietnamese phrases, career transitions, and context beyond term overlap remain limitations. The displayed percentage is a normalized similarity value, not a calibrated probability of recruitment success.

### 6.2.4 Feedback Learning

The system records GOOD_MATCH, POTENTIAL, BAD_MATCH, and NOT_INTERESTED feedback with actor and channel information. Positive and negative learning signals update the Job representation using Rocchio with $\alpha=1.0$, $\beta=0.75$, and $\gamma=0.15$. Recalculation begins from the original Job vector and current feedback history, which supports idempotent metric output. Affected Matchings are marked stale and rescored asynchronously.

The controlled benchmark confirms the intended Rocchio behavior in the planted latent-skill scenario. The final run completed without the earlier optimistic-lock exception after learning was moved to an after-commit boundary and benchmark

state was cleared before recomputation. nDCG@5 and HitRate@5 each increased by 0.80, although this does not prove reliability under production concurrency.

### 6.2.5 Policy-Driven Automation and Email Action

AutoFit policies store thresholds, enablement, notification preferences, cooldown, quota, quiet-hour, timezone, digest, and pause settings. Scheduled services recompute stale results, send digests and high-match notifications, clean expired email actions, and execute auto-apply. Auto-apply validates the default CV, Job status, threshold, and duplicate state and limits creation to three applications per run.

This objective is achieved as configurable prototype automation. Notification policy and application authorization remain separate control paths, scheduler timing is externalized through app.scheduler.*, and the email-action channel uses hashed token storage with GET confirmation followed by POST execution. Further production work is still required for rate limiting, secret rotation, mail delivery, and deployment-specific origin controls.

### 6.2.6 Auditability and Evaluation

Major application, feedback, automation, and administrative actions create structured audit rows. Notification delivery and email-action statuses add operational evidence. The project includes unit, integration, security, algorithm, and E2E tests and produces a dataset-hashed benchmark artifact.

Audit coverage has not been formally proven for every state-changing endpoint, and direct repository calls can still produce inconsistent metadata conventions. The evaluation nevertheless checked logs and side effects in addition to test exit codes, which helped detect and correct the earlier concurrency and health problems.

*Table 6.2. Objective assessment*

| Objective | Assessment | Primary evidence |
|---|---|---|
| Role-based web platform | Achieved for prototype scope | API contracts and 20 Chromium workflow, contract, and resilience tests |
| CV/JD ingestion and validation | Achieved for supported formats and configured OCR | Extraction, validation, Job, and integration tests |
| Explainable lexical matching/recommendation | Achieved as baseline | TF-IDF/scoring tests and implemented reasons |

| Objective | Assessment | Primary evidence |
|---|---|---|
| Rocchio feedback adaptation | Achieved in the controlled test; production concurrency unverified | Current synthetic benchmark, after-commit execution, and clean background log |
| AutoFit and actionable communication | Achieved as a configurable prototype | Auto-apply tests, scheduler, immediate notifications, and hashed confirm-then-POST email actions |
| Security and auditability | Partially verified | Security tests and spot checks; incomplete penetration/audit coverage |
| Production readiness | Not demonstrated | Local health passed; scale, independent users, and production security evidence remain missing |

## 6.3 Discussion

### 6.3.1 Value of an Interpretable Baseline

CareerFit deliberately uses a lexical vector-space baseline rather than presenting a dense or generative model as inherently superior. This choice makes the input tokens, weights, shared terms, feedback centroids, and score calculation inspectable. For an academic system, that transparency supports explanation, debugging, boundary testing, and defense of the implementation.

The cost is reduced semantic capability. A model that cannot reliably equate related technologies or interpret career context may under-rank suitable candidates and over-rank documents containing repeated keywords. The correct conclusion is not that TF-IDF is sufficient for all recruitment, but that it provides a reproducible baseline against which future semantic or hybrid retrieval can be compared.

### 6.3.2 Human-in-the-Loop as System Structure

Human-in-the-Loop in CareerFit is expressed through multiple control points: users supply and validate data, inspect reasons, configure policy, apply or invite, provide feedback, pause automation, and review audit history. The system distinguishes score evidence from Application state and policy action. This is more meaningful than adding a generic approval button after an otherwise opaque automated decision.

Human oversight nevertheless does not automatically make a system fair or safe. Users may accept misleading explanations, policies may contain inappropriate thresholds, and feedback can reproduce individual or institutional bias. HITL must therefore be accompanied by measured error analysis, access control, contestability, audit review, and representative evaluation.

### 6.3.3 Meaning of the Rocchio Improvement

The large benchmark delta is expected from the experimental design: a latent skill is deliberately shared between a training-positive CV and a holdout-positive CV but omitted from the initial JD emphasis. Rocchio moves the Job vector toward the training example, making the holdout example easier to retrieve. Repeated equality of the metrics shows implementation determinism under this scenario.

The result does not show that every real Candidate feedback event improves ranking by 0.80 nDCG@5. Organic feedback may be inconsistent, sparse, biased, delayed, or based on salary and location rather than technical relevance. Production evaluation would require independently labeled data, time-based splits, multiple reviewers, disagreement analysis, and online or offline comparison against a baseline without feedback.

### 6.3.4 Green Assertions versus Operational Cleanliness

The earlier background StaleObjectStateException demonstrated why test counts cannot be the only release criterion. JUnit assertions can pass while asynchronous tasks log failures outside the foreground test lifecycle. Likewise, component-level and aggregate health must be interpreted together. A credible system report must examine logs, health components, artifacts, and side effects in addition to process exit codes.

Future CI should continue to fail on uncaught background exceptions, control schedulers during algorithm tests, wait for asynchronous work explicitly, and archive logs as first-class results. Operational health details should be protected while remaining available to authorized operators.

#### 6.3.5 Product Positioning

CareerFit should be positioned as an IT-focused, explainable, policy-controlled academic recruitment prototype. It demonstrates how Job portal functions, ranking, recommendation, feedback, automation, email actions, and audit can be connected. It should not be positioned as having better general-purpose AI than commercial recruiter platforms or as replacing professional recruitment judgment.

The defensible differentiation is workflow control: matching and recommendation are separate, policy is explicit, users retain control points, feedback has a visible mathematical effect, and actions can be audited. This differentiation is narrower but better supported by the implementation.

### 6.4 Limitations

#### 6.4.1 Data and Algorithm Limitations

The controlled dataset is synthetic and intentionally structured to demonstrate causal learning. It lacks natural language variation, recruiter disagreement, demographic analysis, adversarial CVs, and changing labor-market distributions. The runtime database contains seed and scraped Jobs but no validated relevance ground truth. TF-IDF uses a hand-curated static IT corpus, whitespace tokenization, and maximum IDF for unknown terms. Potential detection uses fixed shared-term and seniority heuristics.

#### 6.4.2 Evaluation Limitations

The 72 backend tests do not prove exhaustive path coverage. Browser evaluation covers 20 automated cases in Chromium only, and no independent users participated. Security checks were targeted API observations rather than penetration testing, while the latency sample used 30 sequential warm requests on one workstation. Aggregate health passed in the final run, but concurrency, capacity, cross-browser behavior, and real-user usability remain unevaluated.

### 6.4.3 Security and Privacy Limitations

Notification action tokens are hashed and state changes require POST after confirmation. Access tokens are stored in frontend sessionStorage rather than persistent localStorage. Local CV storage lacks documented malware scanning, encryption, retention enforcement, and recovery testing. Public Swagger and Prometheus access are suitable for local demonstration but should be restricted in production. Privacy, fairness, and data-subject governance have not been validated with a real deployment population.

### 6.4.4 Reliability and Maintainability Limitations

The final remediation pass resolved the observed optimistic-lock benchmark exception, scheduler/configuration drift, mock Job fallback values, persistent localStorage authentication, oversized frontend bundle warning, and missing E2E Job cleanup. Audit records are still written directly by multiple services, and the evaluated worktree is not an immutable release commit; those limitations weaken production operations and exact source-to-evidence reproducibility.

*Table 6.3. Principal limitations and impact*

| Limitation | Impact on conclusions |
|---|---|
| Synthetic benchmark | Supports causal behavior only, not real hiring effectiveness |
| Lexical representation | Limits synonym, semantic, and context understanding |
| Production concurrency not evaluated | Final benchmark is clean, but production conflict and retry behavior remain unproven |
| Local-only health verification | Health passed locally but does not establish deployed availability |
| Chromium-only 20-test automation suite | Limits cross-browser and real-user usability conclusions |
| Email delivery and abuse controls not production-tested | Hashed confirm-then-POST flow is implemented; rate limiting and real delivery remain unverified |
| Small localhost latency sample | Cannot establish scale, throughput, or cloud performance |
| Dirty worktree and mutable E2E database | Weakens exact experiment reproduction |

## 6.5 Future Work

Future security work should add rate limiting, stronger session handling, protected management endpoints, malware scanning, encrypted CV storage, retention rules, and tested backup recovery. Email delivery should be tested with a real provider, including expiry, replay, scanner behavior, and failure recovery. CI should continue checking background logs and asynchronous completion rather than relying only on foreground test status.

The matching baseline can then be extended using a hybrid approach. Domain-aware tokenization and aliases should first address punctuation-sensitive technologies and Vietnamese phrases. Sparse lexical scores can be combined with sentence or skill embeddings and structured constraints. Any new model should be evaluated against the current baseline on a frozen, independently labeled dataset rather than assumed superior because it is more complex.

Evaluation should expand to recruiter-labeled CV–JD pairs, temporal holdout sets, inter-rater agreement, hard-negative analysis, subgroup/fairness checks, and calibration of labels. A controlled user study should measure task completion, time, comprehension of explanations, trust calibration, and ability to override automation. Cross-browser E2E, load testing, failure injection, backup/restore, email delivery, and security testing should be included in release evidence.

Operational development should move CVs to protected object storage, define encryption and retention policy, centralize audit generation and redaction, strengthen token/session architecture, and associate each release with a clean commit and immutable evidence archive. Integration with external ATS or Job boards should occur only after consent, ownership, error recovery, and audit contracts are defined.

## 6.6 Conclusion

CareerFit demonstrates an end-to-end approach to controlled recruitment automation in the IT domain. It connects job discovery, interpretable matching, profile-based recommendation, feedback learning, policy-driven actions, and audit records while keeping the relationship between score, business state, and user decision visible.

The final evidence shows that the prototype works in the evaluated local environment: backend tests passed, the frontend built successfully, selected browser

workflows completed, aggregate health was UP, and Rocchio produced deterministic improvement in the synthetic scenario. The main contribution is therefore a working and critically evaluated Human-in-the-Loop workflow, not a claim that the model is universally superior or ready to make production hiring decisions.

# REFERENCES


[1] G. Salton, A. Wong, and C. S. Yang, “A vector space model for automatic indexing,” Communications of the ACM, vol. 18, no. 11, pp. 613–620, 1975, doi: 10.1145/361219.361220.

[2] C. D. Manning, P. Raghavan, and H. Schütze, Introduction to Information Retrieval. Cambridge, U.K.: Cambridge University Press, 2008.

[3] C. Buck and P. Koehn, “Quick and reliable document alignment via TF/IDF-weighted cosine distance,” in Proceedings of the First Conference on Machine Translation, vol. 2, Berlin, Germany, 2016, pp. 672–678, doi: 10.18653/v1/W16-2365.

[4] J. J. Rocchio, “Relevance feedback in information retrieval,” in The SMART Retrieval System: Experiments in Automatic Document Processing, G. Salton, Ed. Englewood Cliffs, NJ, USA: Prentice-Hall, 1971, pp. 313–323.

[5] K. Järvelin and J. Kekäläinen, “Cumulated gain-based evaluation of IR techniques,” ACM Transactions on Information Systems, vol. 20, no. 4, pp. 422–446, 2002, doi: 10.1145/582415.582418.

[6] N. Drushchak and M. Romanyshyn, “Introducing the Djinni Recruitment Dataset: A corpus of anonymized CVs and job postings,” in Proceedings of the Third Ukrainian Natural Language Processing Workshop, Torino, Italy, 2024, pp. 8–13, doi: 10.63317/2dcvy45ws6yb.

[7] X. Yu, R. Xu, C. Xue, J. Zhang, X. Ma, and Z. Yu, “ConFit v2: Improving resume-job matching using hypothetical resume embedding and runner-up hard-negative mining,” in Findings of the Association for Computational Linguistics: ACL 2025, Vienna, Austria, 2025, pp. 12775–12790, doi: 10.18653/v1/2025.findings-acl.661.

[8] S. Vaishampayan et al., “Human and LLM-based resume matching: An observational study,” in Findings of the Association for Computational Linguistics: NAACL 2025, Albuquerque, NM, USA, 2025, pp. 4823–4838, doi: 10.18653/v1/2025.findings-naacl.270.

[9] E. Tabassi, Artificial Intelligence Risk Management Framework (AI RMF 1.0), NIST AI 100-1. Gaithersburg, MD, USA: National Institute of Standards and Technology, 2023, doi: 10.6028/NIST.AI.100-1.

[10] M. Raghavan, S. Barocas, J. Kleinberg, and K. Levy, “Mitigating bias in algorithmic hiring: Evaluating claims and practices,” in Proceedings of the 2020 Conference on Fairness, Accountability, and Transparency, Barcelona, Spain, 2020, pp. 469–481, doi: 10.1145/3351095.3372828.

[11] R. T. Fielding, “Architectural styles and the design of network-based software architectures,” Ph.D. dissertation, University of California, Irvine, CA, USA, 2000.

[12] M. Jones, J. Bradley, and N. Sakimura, “JSON Web Token (JWT),” Internet Engineering Task Force, RFC 7519, May 2015, doi: 10.17487/RFC7519.

[13] K. Kent and M. Souppaya, Guide to Computer Security Log Management, NIST SP 800-92. Gaithersburg, MD, USA: National Institute of Standards and Technology, 2006, doi: 10.6028/NIST.SP.800-92.

[14] A. Colas, J. Araki, Z. Zhou, B. Wang, and Z. Feng, “Knowledge-grounded natural language recommendation explanation,” in Proceedings of the 6th BlackboxNLP Workshop, Singapore, 2023, pp. 1–15, doi: 10.18653/v1/2023.blackboxnlp-1.1.

# APPENDICES

## Appendix A. Requirement–Test Traceability Matrix

The core traceability chain is: authentication and role controls → SecurityHardeningTest and P0 login flows; CV ingestion → service and integration tests plus the Candidate upload flow; matching and feedback → AlgorithmEvaluatorTest; Job lifecycle → recruiter P0 create/verify/delete flow; administration → suspend/activate P0 flow; operations → Actuator health and build evidence.

## Appendix B. Selected API Contracts

Representative endpoints are POST /api/auth/login; GET and POST /api/jobs; POST /api/cvs/upload; GET /api/matchings; POST /api/feedback; GET and PATCH /api/automation/policies/me; GET then POST /api/email-action/redeem; and GET /actuator/health. Protected endpoints require a signed JWT and enforce role or ownership rules in the backend.

## Appendix C. Data Model Summary

Identity data is centered on Account and role-specific profiles. Recruitment data includes Job, CV, Matching, Application, Invitation, and Feedback. Automation and operations use AutomationPolicy, EmailAction, EmailToken, Notification, AuditLog, and scheduler state. Flyway migrations provide the reproducible schema history; action tokens are persisted as SHA-256 hashes.

## Appendix D. Evaluation Summary

The final evidence package contains the complete backend test log, isolated algorithm benchmark, frontend production build, browser-based P0 results, runtime health response, screenshots, and evaluation/result.json. Chapter 5 reports the exact observed results and limitations; these artifacts support demo and thesis-defense readiness rather than a production-deployment claim.

## Appendix E. UAT and Demonstration Script

1) Search for a public IT Job and open its details. 2) Sign in as Candidate, upload/select a CV, inspect ranked matches and explanations, apply, and withdraw. 3) Sign in as Recruiter, create a Job, inspect candidate ranking, and remove test data. 4) Configure AutoFit and explain threshold, quota, cooldown, and human override. 5) Sign in as Administrator, suspend and reactivate a test account, then inspect the audit view. 6) Show Actuator health and the archived test evidence.

## Appendix F. Local Deployment Instructions

Prerequisites are Java 21, Node.js with npm, Docker Desktop, and Git. Start PostgreSQL with docker compose up -d postgres. From Backend/careerfit-backend, run mvnw.cmd spring-boot:run. From Frontend, run npm install and npm run dev. Host-mode PostgreSQL is localhost:5433; services inside the Compose network use postgres:5432. Verify GET http://localhost:8080/actuator/health and open http://localhost:5173. Secrets and production controls must be supplied through environment-specific configuration.